

MIRROR
An AI Powered Memory and Emotion Analytics
System
OLMCA699 - Dissertation

Submitted in partial fulfillment of the requirements for the degree of

Master of Computer Applications
in
Department of Computer Applications

by
Prabhu Teja Pamula
24EMCA1237

Under the guidance of
Prof./Dr. BRIJENDRA SINGH

School of Computer Science and Engineering
VIT,Vellore



June, 2026

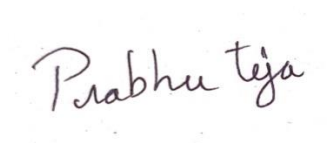
DECLARATION

I **Prabhu Teja Pamula** here by declare that the dissertation entitled “**MIRROR: An AI Powered Memory and Emotional Analytics System**” submitted by me, for the award of the degree of *Master of Computer Applications in Department of Computer Applications, School of Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of **Dr. Brijendra Singh, Associate Professor, SCOPE, VIT, Vellore.**

I further declare that the work reported in this dissertation has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 30-06-2026

A handwritten signature in dark ink, reading "Prabhu Teja", is written over a faint, circular, textured background.

Signature of the Candidate

CERTIFICATE

This is to certify that the dissertation entitled “**MIRROR: An AI Powered Memory and Emotional Analytics System**” submitted by **Prabhu Teja Pamula & 24EMCA1237**, SCOPE, VIT, for the award of the degree of *Master of Computer Applications in Department of Computer Applications*, is a record of bonafide work carried out by him under my supervision during the period, 27/03/26 to 03/07/26, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The dissertation fulfills the requirements and regulations of the University and in my opinion meet the necessary standards for submission.

Place: Vellore

Date: 30-06-2026

Signature of the VIT-SCORE/SCOPE - Guide

Head of the Department
Department of Computer Applications

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my dissertation guide **Dr. Brijendra Singh, Associate Professor**, School of Computer Science and Engineering Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Artificial Intelligence and Emotional Analytics.

I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Dr. Sankar Viswanathan, Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T.**

My whole hearted thanks to Director, VITOL, School of Computer Science and Engineering Head, Department of Computer Applications, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my dissertation successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the dissertation.

Place: Vellore

Date: 30-06-2026

Prabhu Teja Pamula

Executive Summary

In today's fast paced world, personal insights and emotional reflections often decay rapidly, lost to time and the friction of maintaining static journals. MIRROR is an innovative AI powered memory and emotional analytics platform designed to solve this problem by transforming how individuals capture, process and engage with their experiences.

By offering a low friction, multimodal interface, MIRROR allows users to effortlessly record their thoughts through direct text or voice inputs. Behind the scenes, the platform leverages advanced Large Language Models and high dimensional semantic search to analyze these entries. Instead of merely storing data, MIRROR actively synthesizes emotional metadata and uncovers hidden behavioral patterns over time.

The system is built on a scalable, secure and cloud native microservices architecture, ensuring robust performance alongside strict privacy for sensitive user information. Through asynchronous processing and dynamic data retrieval, the platform guarantees a highly responsive, seamless user experience across all modern devices.

Ultimately, MIRROR successfully elevates the concept of digital journaling from passive record keeping to proactive self discovery. By mirroring past experiences into meaningful and actionable insights, the platform empowers everyday users to effectively cultivate deeper self awareness, reinforce positive habits and sustain lifelong continuous personal development alongside psychological well being.

Table of Contents

Contents	Page No.
DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
Executive Summary	v
Table of Contents	vi
List of Figures	x
List of Tables	xii
List of Abbreviations	xiii
Symbols and Notations	xv
1 INTRODUCTION	1
1.1 Objective	1
1.2 Motivation	2
1.3 Background	3
2 DISSERTATION DESCRIPTION AND GOALS	4
2.1 System Overview	4
2.2 Functional Requirement	5
2.3 Non - Functional Requirement	7
2.4 Project Goals	8
3 TECHNICAL SPECIFICATION	10
3.1 Implementation Environment	10
3.1.1 Development Stack and Prerequisites	10

3.1.2	Local Infrastructure Standardization (Docker)	10
3.1.3	CI/CD and Version Control Strategy	11
3.2	Backend Services Architecture	15
3.2.1	Auth Service: Security and Identity Management	15
3.2.2	Memory Service: The Core Intelligence Pipeline	16
3.2.3	Asynchronous Event Handling (RabbitMQ)	16
3.3	Data Layer Engineering	16
3.3.1	Relational Schema and Vector Storage (PostgreSQL & pgvector)	16
3.3.2	Caching Strategies and Redis Integration	17
3.2.4	Data Consistency and Integrity Mechanisms	17
3.4	API Gateway and Security Infrastructure	18
3.4.1	Gateway Routing and Traffic Management	18
3.2.5	Resilience Patterns: Circuit Breaking and Rate Limiting	18
3.5	Observability and Distributed Tracing	19
3.5.1	Centralized Logging and Performance Monitoring (Zipkin)	19
3.5.2	Debugging and Latency Analysis	19
3.6	Integration with External AI Services	20
3.6.1	Embedding Generation (Gemini API)	20
3.6.2	Real-Time Inference and Latency Optimization (Groq API)	20
3.6.3	Third-Party Communication Services (Brevo SMTP)	20
4	DESIGN APPROACH AND DETAILS	21
4.1	Design Approach / Materials & Methods	21
4.1.1	Architectural Paradigm	21
4.1.2	The Technology Stack Selection	21
4.1.3	Development Methodology	22
4.2	Codes and Standards	23

4.2.1	Security Standards (OWASP Compliance)	23
4.2.2	Coding Standards and SOLID Principles	24
4.2.3	Data and Communication Standards	24
4.3	Constraints, Alternatives, and Tradeoffs	24
4.3.1	Performance Constraints and Latency Tradeoffs	24
4.3.2	Infrastructure and Scalability	25
4.3.3	Retrieval-Augmented Generation (RAG) vs. Model Fine-Tuning	25
5	SCHEDULE, TASKS AND MILESTONES	27
5.1	Project Lifecycle Methodology	27
5.2	Milestone Matrix	27
5.3	Task Allocation and Resource Management	28
5.4	Retrospective Analysis	29
6	DISSERTATION DEMONSTRATION	30
6.1	Sample Codes	30
6.1.1	Authentication and Identity Management Phase	30
6.1.2	Data Engineering and Vector Storage Phase	42
6.1.3	Artificial Intelligence and Cognitive Processing Phase	51
6.1.4	Cloud Infrastructure and Resilience Phase	94
6.1.5	Client Application and Hardware Integration Phase	97
6.2	Sample Screen Shots	113
6.2.1	Authentication & Onboarding Views	113
6.2.2	Main Dashboard & Interaction Views	115
6.2.3	Administrator Views	119
6.2.4	Application Exit	124

7	COST ANALYSIS, RESULT AND DISCUSSION	125
7.1	Cost Analysis	125
7.2	Results and Discussion	127
8	SUMMARY	129
8.1	Project Overview	129
8.2	Achievement of Objectives	129
8.3	Limitations and Challenges	130
8.4	Future Research Directions	131
9	REFERENCES	132
	APPENDIX A	134
A.1	System Deployment Guide	134
A.2	Core AI Database Schema	135
A.3	Microservice API Routing Table	135
A.4	System Access & Observability	136

List of Figures

Figure No.	Title	Page No.
2.1	High level architectural overview.	5
3.1	Overview of the MIRROR Backend Services Architecture.	15
4.1	RAG-based Sequence Diagram for Cognitive Reflection.	26
6.2.1	User Login Interface.	113
6.2.2	Account Registration.	114
6.2.3	OTP Verification.	114
6.2.4	Password Reset Request Interface.	115
6.2.5	Ambient Welcome Screen.	115
6.2.6	Chat interface with Daily Quote for first-time users or when a new day starts.	116
6.2.7	Active Chat Interface.	116
6.2.8	Voice Transcription and Chat History.	117
6.2.9	Expanded Sidebar Navigation Menu.	117
6.2.10	Mood and Emotional State Tracker.	118
6.2.11	User Profile and Account Settings.	118
6.2.12	Project Acknowledgements and Appreciation Modal.	119
6.2.13	Administrator Profile Interface.	119
6.2.14	Admin Service Selection Interface.	120
6.2.15	User Management Dashboard.	120
6.2.16	Create New User Modal Interface.	121
6.2.17	Edit User Account Modal Interface.	121
6.2.18	Memory Service Database Management	122

	Dashboard.	
6.2.19	Create New Memory Record Modal.	122
6.2.20	Expanded User Memory Records View	123
6.2.21	Edit Memory Record Modal	123
6.2.22	Memory Data Bulk Upload Interface	124
6.2.23	Log Out Confirmation Modal	124

List of tables

Table No.	Title	Page No.
2.1	Functional Requirement	06
2.2	Non Functional Requirement	07
4.1.2	The Technology Stack Selection	22
5.2	Milestone Matrix	27
7.1	Cost Analysis	
A.3	Microservice API Routing Table	135

List of Abbreviations

AI	Artificial Intelligence
HNSW	Hierarchical Navigable Small World
IVFFlat	Inverted File with Flat Residuals
K-NN	k-Nearest Neighbors
LLM	Large Language Model
LPU	Language Processing Unit
RAG	Retrieval-Augmented Generation
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DDoS	Distributed Denial of Service
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
I / O	Input / Output
JSON	JavaScript Object Notation
JWT	JSON Web Token
NFR	Non-Functional Requirement
OPEX	Operational Expenditure
ORM	Object-Relational Mapping
PWA	Progressive Web Application
RBAC	Role-Based Access Control
REST	Representational State Transfer
SOLID	Single responsibility, Open-closed, Liskov substitution, Interface segregation, and

	Dependency inversion
SRP	Single Responsibility Principle
TLS	Transport Layer Security
OTP	One-Time Password
URI	Uniform Resource Identifier
UUID	Universally Unique Identifier
APK	Android Package
CI/CD	Continuous Integration and Continuous Delivery
DIP	Dependency Inversion Principle
HikariCP	Hikari Connection Pool
JDK	Java Development Kit
JPA	Jakarta Persistence API
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TTS	Text-to-Speech

Symbols and Notations

<code><=></code>	Cosine distance operator
<code>@PreAuthorize</code>	Spring Security method-level role authorization annotation
<code>@Async</code>	Spring framework annotation for non-blocking background thread execution
<code>@RabbitListener</code>	Spring AMQP directive binding background worker threads to processing queues
<code>@Cacheable</code>	Spring cache directive routing lookup queries to memory buffers instead of disk blocks
<code>\$..</code>	Standard syntax environment variable wrapper used in application.yaml configuration scripts
<code>k</code>	The algorithmic variable indicating the number of nearest contextual database items retrieved during a query lifecycle

CHAPTER 1

INTRODUCTION

Journaling has long been a cornerstone of self reflection, evolving from physical diaries to modern note-taking platforms. However, the fundamental limitations of these solutions persist. While applications like Evernote or Google Keep provide ubiquitous access, they function as "Static Silos." Information is ingested linearly and stored in isolation, rendering them reactive repositories rather than proactive cognitive partners. Furthermore, such software lacks emotional contextualization, failing to mimic the natural biological process which prioritizes highly salient events. Consequently, profound behavioral insights and personal breakthroughs are often lost to cognitive decay.

To address this gap, the MIRROR platform represents a paradigm shift from passive archiving to an active, AI driven emotional analytics engine. By leveraging a microservices based architecture, MIRROR processes, stores and synthesizes multimodal inputs. Integrating advanced Large Language Models (LLMs) with high dimensional vector databases establishes a "living" context of an individual's history. This ensures that crucial personal milestones are protected against forgetting and proactively surfaced precisely when relevant, transforming raw text into actionable self awareness.

1.1 Objective

The primary goal of this initiative is to architect and deploy MIRROR, facilitating lifelong personal development through automated emotional intelligence. Key deliverables include:

- Building a unified cross platform frontend using Ionic and Angular for seamless Web and Android accessibility.
- Constructing a secure backend with Java Spring Boot and Spring Cloud (featuring an API Gateway) to orchestrate ingestion, authentication and routing.
- Designing an asynchronous, event driven pipeline via RabbitMQ to decouple intensive computational workloads (e.g., vector generation) from client facing threads, guaranteeing high responsiveness.

- Deploying PostgreSQL with the pgvector extension for semantic search, mathematically correlating current entries with historical milestones.
- Engineering a Retrieval Augmented Generation (RAG) workflow to grant the language model permanent access to individual histories, bypassing standard context limits.
- Incorporating robust observability and caching mechanisms via Zipkin for distributed tracing and Redis for state management.
- Creating an Interactive Growth Dashboard that visualizes affective trends, semantic clusters and behavioral patterns, offering continuous, data driven personal guidance.

1.2 Motivation

MIRROR's conception stems from the frailties of human cognition, the shortcomings of existing digital diaries, and the technical hurdles of modern AI processing:

- Neurological mechanisms naturally favor recent stimuli, causing profound realizations to fade rapidly within days of their occurrence.
- Standard messaging and journaling platforms operate as isolated fragments; they capture text but cannot interconnect entries into a cohesive psychological narrative.
- Standard generative models exhibit "Semantic Decay" when handling extensive chronological timelines, losing the continuity essential for accurate life story analytics.
- Monolithic software designs fail under the high latency demands of simultaneous LLM inference and vector querying, severely impacting interface fluidity.
- Therefore, a critical demand exists for an intelligent engine capable of resurfacing pertinent past experiences without introducing operational bottlenecks, ultimately reinforcing positive behavioral shifts.

1.3 Background

The theoretical foundation of this research spans four primary domains: cognitive psychology, information science, artificial intelligence and distributed software architecture.

- **Cognitive Psychology (Emotional Tagging):** Scientific consensus shows the brain favors emotionally charged events. The platform mimics this biological mechanism by automatically appending affective metadata to journal entries, safeguarding pivotal moments from being obscured by mundane records.
- **Information Science (Semantic Vector Space):** Traditional keyword searches frequently miss underlying intent. By adopting high dimensional embeddings, the application mathematically calculates the conceptual proximity between new thoughts and past lessons, facilitating highly accurate contextual retrieval.
- **Artificial Intelligence (Retrieval-Augmented Generation):** Base LLMs are constrained by finite context windows and possess no persistent memory. To solve this, a RAG pipeline integrates the language model with a secure, long term database, establishing an unbroken and coherent chronological persona.
- **Distributed Software Architecture:** Synchronous execution of intensive machine learning tasks inherently degrades application performance. Implementing message brokers and distributed tracing within a microservices ecosystem guarantees that complex operations, like semantic clustering, run asynchronously, preserving seamless scalability and frontend fluidity.

CHAPTER 2

DISSERTATION DESCRIPTION AND GOALS

This chapter provides a comprehensive technical overview of the MIRROR platform, detailing the architectural design, core functional requirements and the specific goals governing its development. It outlines the transition from conceptual objectives to a robust, microservices based implementation, establishing the technical framework for the subsequent system analysis and performance evaluation.

2.1 System Overview

The MIRROR platform is a distributed ecosystem aimed at digitizing and analyzing human reflection. By employing a decoupled, microservices oriented approach, the architecture ensures high availability, scalability and independent deployment of critical components. This structure facilitates the seamless integration of heavy, asynchronous AI workloads, such as semantic embedding generation, without compromising interface responsiveness.

The system architecture is structured across five primary layers:

- **Client Layer:** A unified Angular and Ionic application providing a consistent, cross platform interface for both Web and Android users.
- **Gateway Layer:** A centralized Spring Cloud API Gateway that handles request routing, Cross Origin Resource Sharing (CORS), and Redis backed rate limiting, acting as the secure entry point for all frontend requests.
- **Microservices Layer:** A collection of specialized Spring Boot microservices. Specifically, the Auth Service manages user identity, security and communications, while the Memory Service acts as the core engine handling user reflections, emotional analysis, and Retrieval-Augmented Generation (RAG). These services communicate asynchronously via a RabbitMQ event bus to maintain system decoupling and performance.
- **Infrastructure & Data Layer:** A hybrid storage environment utilizing PostgreSQL

with the pgvector extension for relational data and high-dimensional semantic search, alongside Redis for rapid caching and RabbitMQ as the core message broker for event driven communication. This layer also includes Zipkin for comprehensive distributed tracing across the ecosystem.

- **External APIs Layer:** Integration with powerful third party services, including the Gemini API for generating embeddings, the Groq API for ultra fast Large Language Model (LLM) inference and Brevo SMTP for secure email delivery.

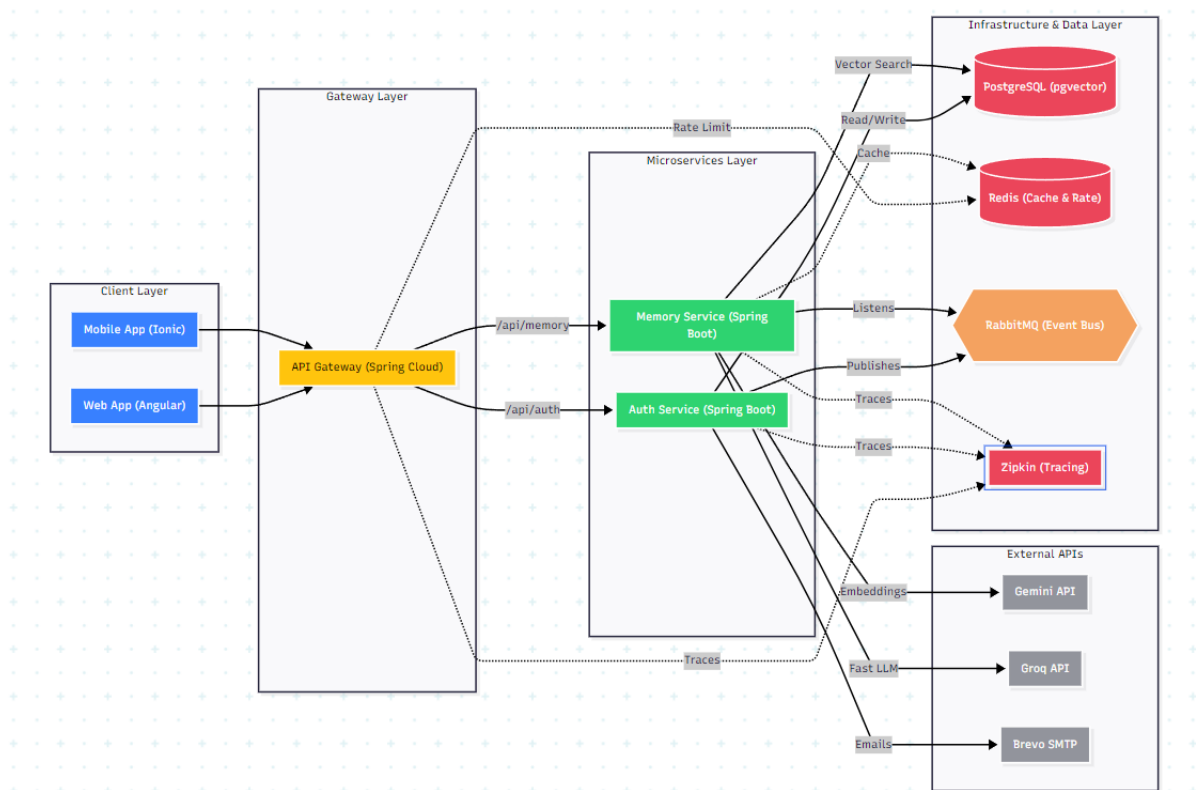


Figure 2.1: High level architectural overview of the MIRROR ecosystem, detailing the interaction between the frontend, API Gateway, microservices and the backend data/AI layers.

2.2 Functional Requirements

The functional requirements define the explicit, user-facing behaviors and core capabilities that the MIRROR platform must deliver to successfully digitize and analyze human reflection. These requirements translate the conceptual goals of the project into concrete,

actionable operations, such as secure authentication, semantic memory ingestion, and AI-driven contextual retrieval. To ensure strict alignment with the microservices architecture detailed in the previous section, each functional requirement is explicitly mapped to the specific backend service responsible for its execution.

ID	Features	Description	Target Service
FR-01	User Authentication & RBAC	Secure account creation, JWT based login, Role-Based Access Control and email verification.	Auth Service
FR-02	Profile & Account Management	Enables users to update personal settings, manage preferences, and initiate password recovery flows.	Auth Service
FR-03	Multi Modal Submission	Allows users to submit daily reflections via traditional text input or real-time voice-to-text recognition.	Memory Service
FR-04	Semantic Embedding Generation	Automatically converts raw text reflections into high-dimensional vectors via the Gemini API.	Memory Service
FR-05	Contextual RAG Retrieval	Queries the VectorDB to retrieve past memories and synthesizes contextually relevant insights.	Memory Service
FR-06	Real-Time LLM Inference	Interfaces with the Groq API to provide ultra-low latency, conversational AI responses to the user.	Memory Service
FR-07	Admin Dashboard & Moderation	Secure internal interfaces for system administrators to oversee platform activity, review memory entries, and manage user accounts.	Client Layer / Auth Service
FR-08	Cross-Platform Accessibility	Provides a unified, responsive interface operable on both Desktop	Client Layer

		Web and Mobile (Android).	
--	--	---------------------------	--

2.3 Non Functional Requirements

While functional requirements dictate the core user-facing capabilities of the platform, the non-functional requirements (NFRs) establish the vital operational standards necessary for the system to function securely, efficiently, and reliably at scale. Rather than defining specific features, these requirements focus on the underlying infrastructure—such as asynchronous communication and distributed observability—ensuring that the MIRROR ecosystem maintains high availability, robust performance, and data consistency across all microservices.

ID	Features	Description	Target Service
NFR-01	Security & Privacy	All traffic must be encrypted via HTTPS/TLS. Passwords must be hashed via BCrypt. All endpoints secured by Spring Security.	API Gateway / Auth
NFR-02	Performance & Latency	Gateway routing must occur under 200ms. Groq LLM inference must begin streaming rapidly to maintain a real-time conversational feel.	Groq API / Gateway
NFR-03	Resilience & Rate Limiting	The system must prevent DDoS attacks and API abuse by enforcing strict token-bucket rate limiting on incoming requests.	API Gateway / Redis
NFR-04	Reliability & Consistency	Ensures background tasks (like sending emails or indexing embeddings) are never lost, using resilient message queuing.	RabbitMQ
NFR-05	Scalability & Availability	The decoupled architecture must allow individual microservices to scale horizontally under heavy user load	Microservices Layer

		without downtime.	
NFR-06	Observability & Tracing	The system must provide centralized logging and distributed tracing to quickly diagnose bottlenecks and debug cross-service requests.	Zipkin
NFR-07	High-Speed Caching	Frequently accessed data (like user profiles or system prompts) must be cached in-memory to drastically reduce relational database load.	Redis
NFR-08	Maintainability & Modularity	The codebase must adhere to SOLID principles, allowing external APIs (like Gemini or Groq) to be swapped out without breaking core logic.	Entire System

2.4 Project Goals

The development of the MIRROR platform is guided by a set of core objectives that bridge the gap between human cognitive psychology and advanced distributed software engineering. These goals directly align with the requirements established in the previous sections:

- **Mitigating Cognitive Memory Decay:** To build an intelligent, proactive journaling ecosystem that actively combats human memory loss by digitizing, securely storing, and semantically indexing personal reflections for instant, lifelong recall.
- **Breaking "Semantic Silos" via RAG:** To transition away from static, isolated journal entries by employing Retrieval-Augmented Generation (RAG) and high-dimensional vector search (PostgreSQL with pgvector). This allows the system to dynamically synthesize insights and connect emotional patterns across a user's entire history of entries.
- **Frictionless, Multi-Modal Interaction:** To drastically lower the barrier to daily self-reflection by offering ultra-low latency voice-to-text and conversational AI interactions. By leveraging the Groq API, the system is designed to feel like a

responsive, real-time companion rather than a traditional static data-entry form.

- Enterprise-Grade Distributed Resilience: To guarantee zero-data-loss and high availability for deeply personal user data by utilizing a decoupled, event-driven Spring Boot microservices architecture, backed by asynchronous RabbitMQ message brokering and Redis caching.
- Zero-Trust Security & Privacy: To ensure that highly sensitive user reflections are fiercely protected through industry-standard encryption, stateless JWT authentication, and strict request routing enforced by the centralized Spring Cloud API Gateway.

CHAPTER 3

TECHNICAL SPECIFICATION

The transition from architectural design to system implementation requires a rigorous technical framework to ensure the platform's viability, scalability, and maintainability. This chapter details the concrete engineering decisions, development environments, and modular configurations that transform the MIRROR platform's conceptual architecture into a functional, microservices-based ecosystem. By standardizing the development stack and employing enterprise-grade patterns for data management, asynchronous communication, and security, this chapter demonstrates the technical rigor applied to the platform's construction. The subsequent sections explore the specific backend services, data storage engineering, and external AI integrations that collectively enable MIRROR to facilitate complex, AI-driven human reflection.

3.1 Implementation Environment

3.1.1 Development Stack and Prerequisites

The MIRROR platform is built upon a high-performance stack chosen for its reliability and developer productivity. The core backend services are developed using Java 17+ with the Spring Boot 3.x framework. This choice ensures access to the latest security features and native support for modern cloud-native deployment patterns. The frontend is powered by Angular 20 and Ionic, providing a unified codebase for both Web and Android platforms.\

3.1.2 Local Infrastructure Standardization (Docker)

To eliminate "it works on my machine" issues and streamline the onboarding process, the platform's backing infrastructure is containerized for local development. Essential stateful services and brokers, specifically the Redis cache, RabbitMQ broker, Zipkin tracing server, and PostgreSQL database (with pgvector), are provisioned as isolated containers using a unified compose.yaml file. This approach provides developers with a reliable, one-click local environment that accurately mirrors the infrastructure topology required by the microservices, without the overhead of manually installing and configuring these separate

systems on their local machines.

```
services:
  db:
    image: pgvector/pgvector:pg16
    container_name: mirror-vector-db
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-mirror_admin}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mirror_secure_pass}
      POSTGRES_DB: ${POSTGRES_DB:-mirror_memory}
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7
    container_name: mirror-redis
    ports:
      - "6379:6379"

  zipkin:
    image: openzipkin/zipkin
    container_name: mirror-zipkin
    ports:
      - "9411:9411"

  rabbitmq:
    image: rabbitmq:3-management
    container_name: mirror-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"

volumes:
  pgdata:
```

3.1.3 CI/CD and Version Control Strategy

Development is managed through a modular Git repository. The CI/CD pipeline is designed for independent deployment; changes to the Auth Service do not trigger re-builds of the Memory Service, allowing for rapid iteration and reduced regression risk.

```
name: Mirror CI

on:
  push:
    branches:
      - '**'
  pull_request:
    branches:
      - main

jobs:
  frontend-qa:
    runs-on: ubuntu-latest
```

```

steps:
- uses: actions/checkout@v4

- name: Use Node.js 22.x
  uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: 'npm'
    cache-dependency-path: frontend/package-lock.json

- name: Install dependencies
  working-directory: frontend
  run: npm ci

- name: Run Lint
  working-directory: frontend
  run: npm run lint

- name: Run Tests
  working-directory: frontend
  env:
    FIREBASE_API_KEY: ${ secrets.FIREBASE_API_KEY }
    FIREBASE_AUTH_DOMAIN: ${ secrets.FIREBASE_AUTH_DOMAIN }
    FIREBASE_DATABASE_URL: ${ secrets.FIREBASE_DATABASE_URL }
    FIREBASE_PROJECT_ID: ${ secrets.FIREBASE_PROJECT_ID }
    FIREBASE_STORAGE_BUCKET: ${ secrets.FIREBASE_STORAGE_BUCKET }
    FIREBASE_MESSAGING_SENDER_ID: ${ secrets.FIREBASE_MESSAGING_SENDER_ID }
    FIREBASE_APP_ID: ${ secrets.FIREBASE_APP_ID }
    FIREBASE_MEASUREMENT_ID: ${ secrets.FIREBASE_MEASUREMENT_ID }
  run: npm run test -- --watch=false --no-progress --browsers=ChromeHeadlessCI

deploy-vercel:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs: frontend-qa
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v4

  - name: Use Node.js 22.x
    uses: actions/setup-node@v4
    with:
      node-version: 22

  - name: Install dependencies
    working-directory: frontend
    run: npm ci

  - name: Build Production
    working-directory: frontend
    env:
      API_URL: ${ secrets.API_URL }
      FIREBASE_API_KEY: ${ secrets.FIREBASE_API_KEY }
      FIREBASE_AUTH_DOMAIN: ${ secrets.FIREBASE_AUTH_DOMAIN }
      FIREBASE_DATABASE_URL: ${ secrets.FIREBASE_DATABASE_URL }
      FIREBASE_PROJECT_ID: ${ secrets.FIREBASE_PROJECT_ID }
      FIREBASE_STORAGE_BUCKET: ${ secrets.FIREBASE_STORAGE_BUCKET }
      FIREBASE_MESSAGING_SENDER_ID: ${ secrets.FIREBASE_MESSAGING_SENDER_ID }
      FIREBASE_APP_ID: ${ secrets.FIREBASE_APP_ID }
      FIREBASE_MEASUREMENT_ID: ${ secrets.FIREBASE_MEASUREMENT_ID }
    run: npm run build

  - name: Deploy to Vercel
    env:

```

```

    VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
    VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }
    run: npx vercel deploy --prod --token=${ secrets.VERCEL_TOKEN } --yes

build-android:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs: frontend-qa
  runs-on: ubuntu-latest
  permissions:
    contents: write
  steps:
    - uses: actions/checkout@v4

    - name: Use Node.js 22.x
      uses: actions/setup-node@v4
      with:
        node-version: 22

    - name: Install dependencies
      working-directory: frontend
      run: npm ci

    - name: Build Production
      working-directory: frontend
      env:
        API_URL: ${ secrets.API_URL }
        FIREBASE_API_KEY: ${ secrets.FIREBASE_API_KEY }
        FIREBASE_AUTH_DOMAIN: ${ secrets.FIREBASE_AUTH_DOMAIN }
        FIREBASE_DATABASE_URL: ${ secrets.FIREBASE_DATABASE_URL }
        FIREBASE_PROJECT_ID: ${ secrets.FIREBASE_PROJECT_ID }
        FIREBASE_STORAGE_BUCKET: ${ secrets.FIREBASE_STORAGE_BUCKET }
        FIREBASE_MESSAGING_SENDER_ID: ${ secrets.FIREBASE_MESSAGING_SENDER_ID }
        FIREBASE_APP_ID: ${ secrets.FIREBASE_APP_ID }
        FIREBASE_MEASUREMENT_ID: ${ secrets.FIREBASE_MEASUREMENT_ID }
      run: npm run build

    - name: Setup Java Runtime for Android Build
      uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '21'

    - name: Build Android APK via Gradle
      working-directory: frontend
      run: |
        npx cap sync android
        cd android
        chmod +x gradlew
        ./gradlew assembleDebug
        mv app/build/outputs/apk/debug/app-debug.apk app/build/outputs/apk/debug/Mirror.apk

    - name: Upload Mirror.apk to GitHub Releases
      uses: softprops/action-gh-release@v2
      with:
        files: frontend/android/app/build/outputs/apk/debug/Mirror.apk
        tag_name: mirror
        name: Mirror Mobile Build
        make_latest: true
        allow_updates: true

deploy-backend:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs: frontend-qa

```

```

runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4

- name: Set up JDK 21
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: 'maven'

- name: Grant Execute Permission to Maven Wrapper
  run: chmod +x mvnw
  working-directory: backend

- name: Build with Maven
  run: ./mvnw clean package -DskipTests
  working-directory: backend

- name: Deploy Auth Service to Azure
  uses: azure/webapps-deploy@v3
  with:
    app-name: 'mirror-auth-service'
    publish-profile: '${ secrets.AZURE_AUTH_PUBLISH_PROFILE }'
    package: 'backend/auth-service/target/*.jar'

- name: Deploy API Gateway to Azure
  uses: azure/webapps-deploy@v3
  with:
    app-name: 'mirror-api-gateway'
    publish-profile: '${ secrets.AZURE_GATEWAY_PUBLISH_PROFILE }'
    package: 'backend/api-gateway/target/*.jar'

- name: Deploy Memory Service to Azure
  uses: azure/webapps-deploy@v3
  with:
    app-name: 'mirror-memory-service'
    publish-profile: '${ secrets.AZURE_MEMORY_PUBLISH_PROFILE }'
    package: 'backend/memory-service/target/*.jar'

```

3.2 Backend Services Architecture

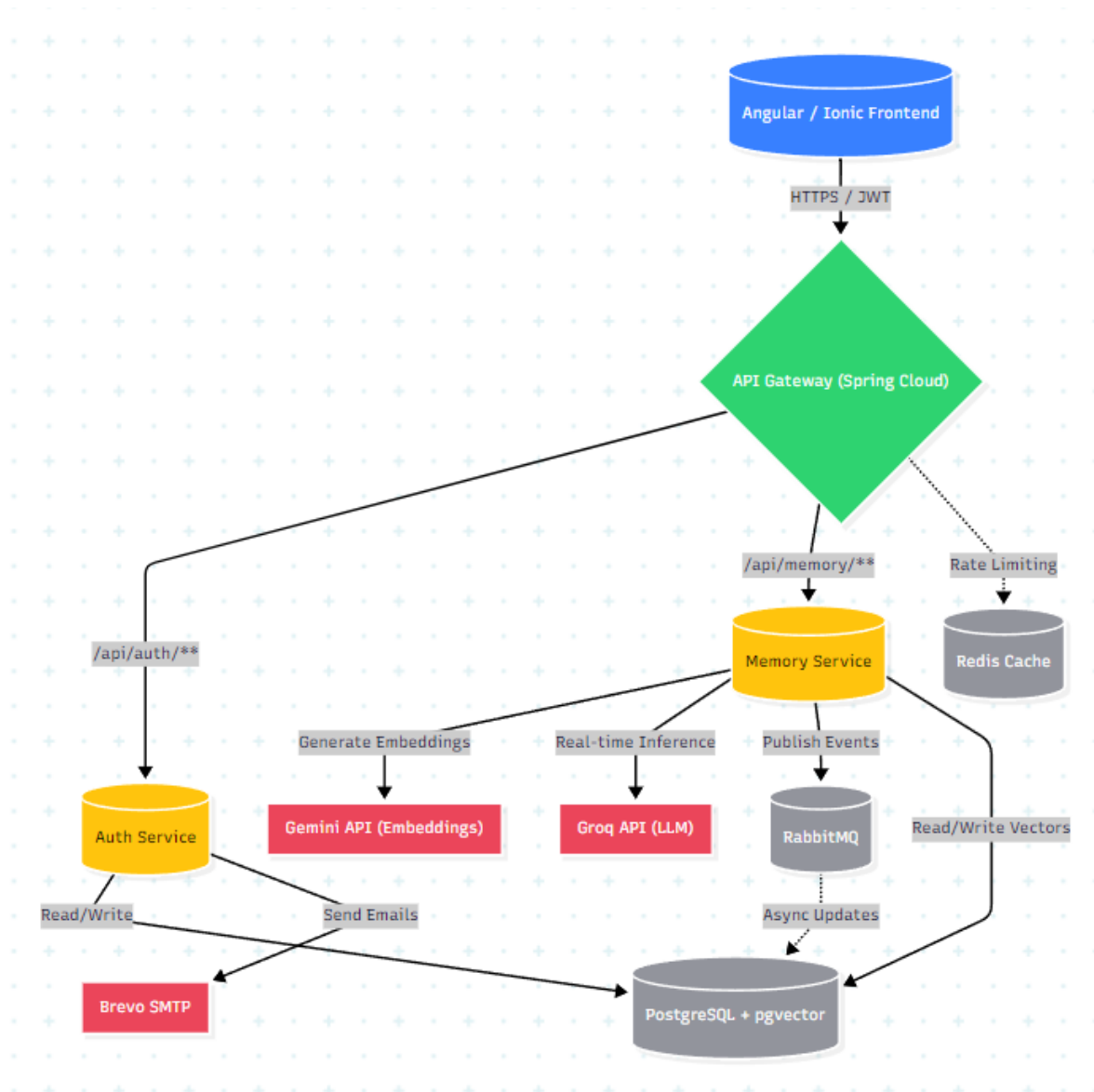


Figure 3.1: Overview of the MIRROR Backend Services Architecture.

3.2.1 Auth Service: Security and Identity Management

- The Auth Service functions as the gatekeeper. It is implemented as a stateless service to ensure it can scale horizontally.
- Security Protocol: We utilize Stateless JWT Authentication. Upon login, a token is issued; the API Gateway then handles token validation to ensure that downstream

services receive verified requests.

- **Credential Handling:** User credentials are protected using BCrypt hashing. This one-way cryptographic function ensures that even in the event of a database compromise, plaintext passwords remain unrecoverable.
- **RBAC Model:** The service employs Role-Based Access Control (RBAC), using Spring Security's method-level annotations (`@PreAuthorize`) to restrict administrative moderation functions to authorized staff.

3.2.2 Memory Service: The Core Intelligence Pipeline

This is the primary engine for cognitive processing. Its implementation focuses on non-blocking I/O to handle heavy AI-driven workloads.

- **RAG Workflow:** The service manages a pipeline that extracts text, cleanses it, and calls the Gemini API for embedding generation.
- **Concurrency:** By using Spring's `CompletableFuture` and asynchronous threads, the service offloads embedding generation to background tasks, keeping the user interface responsive during heavy AI ingestion.

3.2.3 Asynchronous Event Handling (RabbitMQ)

To maintain service decoupling, we implement an event-driven pattern. When a memory is saved, an event is published to a Fanout exchange in RabbitMQ. This allows the Memory Service to commit the data to the database immediately while other services (such as an analytical engine or notification service) process the event independently, ensuring the system remains performant and resilient.

3.3 Data Layer Engineering

3.3.1 Relational Schema and Vector Storage (PostgreSQL & pgvector)

The platform utilizes PostgreSQL not merely as a traditional relational store but as a unified database for both structured metadata and high-dimensional semantic data. By integrating the

pgvector extension, we enable native vector indexing (such as HNSW or IVFFlat) directly within the database engine. This design significantly reduces latency compared to external vector stores, as it eliminates the need for cross-service network round-trips when performing similarity searches.

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
  id UUID PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(255) NOT NULL,
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,
  failed_attempts INT NOT NULL DEFAULT 0,
  locked_until TIMESTAMP
);

CREATE TABLE refresh_tokens (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  token VARCHAR(500) UNIQUE NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);

CREATE TABLE otp_tokens (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  hashed_otp VARCHAR(255) NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  used BOOLEAN NOT NULL DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS memories (
  id BIGSERIAL PRIMARY KEY,
  user_id VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  emotion VARCHAR(50) NOT NULL,
  sender VARCHAR(50) DEFAULT 'user',
  embedding vector(768),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS memories_embedding_idx
ON memories USING hnswn (embedding vector_cosine_ops);
```

3.3.2 Caching Strategies and Redis Integration

To alleviate load on the primary PostgreSQL instance, a "Cache-Aside" pattern is implemented via Redis. Frequently accessed data—including user session configurations, system prompt templates, and hot user profile data—are stored in-memory. This configuration ensures that read-heavy operations, which constitute the majority of API traffic, are served with microsecond latency, thereby preserving database resources for write-

intensive memory ingestion tasks.

3.3.3 Data Consistency and Integrity Mechanisms

Maintaining consistency across a distributed microservices environment requires robust strategies. We employ a combination of local database transactions for atomicity within a single service and the "Transactional Outbox" pattern for cross-service communication. By writing events to an outbox table in the same transaction as business data changes, we ensure that RabbitMQ messages are only published if the database state is successfully updated, effectively mitigating risks associated with partial failures.

3.4 API Gateway and Security Infrastructure

3.4.1 Gateway Routing and Traffic Management

The Spring Cloud API Gateway acts as the unified entry point. It utilizes route predicates to dynamically direct incoming traffic to the appropriate microservice based on URI patterns (e.g., /api/auth/ vs. /api/memory/). Beyond simple routing, the gateway acts as a security enforcement point, centralizing TLS termination and JWT validation, which ensures that internal microservices operate within a trusted network perimeter.

```
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service-route
          uri: http://localhost:8080
          predicates:
            - Path=/api/auth/**

        - id: memory-service-route
          uri: http://localhost:8081
          predicates:
            - Path=/api/memory/**,/api/admin/memory/**
          filters:
            - AuthenticationFilter
```

3.4.2 Resilience Patterns: Circuit Breaking and Rate Limiting

To prevent cascading failures, the gateway integrates Resilience4j.

- **Circuit Breaker:** If the Memory Service becomes unresponsive due to latency in external AI APIs, the circuit breaker trips, providing a fallback response to the user instead of letting threads hang .
- **Rate Limiting:** Using a Redis-backed token bucket algorithm, the gateway limits the number of requests per user identity, protecting the system from brute-force attempts and resource exhaustion.

```
spring:
  cloud:
    gateway:
      routes:
        - id: memory-service-route
          # ... routing configuration omitted ...
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@userKeyResolver}"
            - name: CircuitBreaker
              args:
                name: defaultCircuitBreaker

resilience4j:
  timelimiter:
    instances:
      defaultCircuitBreaker:
        timeoutDuration: 60s
  circuitbreaker:
    instances:
      defaultCircuitBreaker:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 3
```

3.5 Observability and Distributed Tracing

3.5.1 Centralized Logging and Performance Monitoring (Zipkin)

In a distributed microservices environment, diagnosing the source of latency or failures is complex. We have implemented Zipkin for distributed tracing to provide end-to-end visibility of requests. Each request is assigned a unique trace ID at the API Gateway, which propagates through the Auth and Memory services, and into the data layer. This allows for the visualization of the entire request lifecycle, making it straightforward to identify which

service or external API is contributing to request bottlenecks.

3.5.2 Debugging and Latency Analysis

Beyond request tracing, we utilize centralized logging aggregated through a standard logging framework. This structure enables developers to query logs across different services using the trace ID, facilitating rapid debugging of cross-service interactions. This visibility is critical for maintaining the high-performance standards established in our Non-Functional Requirements.

3.6 Integration with External AI Services

3.6.1 Embedding Generation (Gemini API)

The platform integrates the Gemini API as the primary engine for natural language understanding. When a user submits a reflection, the text is sent to Gemini to generate high-dimensional embeddings. These embeddings are the foundation of our semantic search capabilities, allowing the system to understand the underlying context and sentiment of user reflections.

3.6.2 Real-Time Inference and Latency Optimization (Groq API)

To provide a responsive, real-time AI companion experience, we utilize the Groq API for LLM inference. Groq's high-speed Language Processing Units (LPUs) are specifically optimized for low-latency streaming of AI responses. This integration is vital for achieving the sub-second responsiveness required for an engaging user experience, distinguishing the MIRROR platform from systems relying on standard, slower inference endpoints.

3.6.3 Third-Party Communication Services (Brevo SMTP)

System-critical notifications, including email verification and password recovery, are managed via the Brevo SMTP service. This service provides reliable, scalable email delivery, ensuring that user authentication flows—a core functional requirement—operate dependably without the complexity of managing internal mail server infrastructure.

CHAPTER 4

DESIGN APPROACH AND DETAILS

The efficacy of a research-oriented software system is determined not only by its successful implementation but also by the rigor of its design philosophy. While Chapter 3 delineated the technical specifications and infrastructure deployment of the MIRROR platform, this chapter explores the underlying engineering approach, the adherence to industry-standard security and coding practices, and the technical trade-offs necessitated by the system’s design. By documenting the methodologies utilized during the development process and justifying the selection of specific architectural patterns against potential alternatives, this chapter demonstrates the systematic effort to balance performance, scalability, and security. The following sections provide a comprehensive analysis of the standards governing the platform’s construction and the design decisions that define its operational behavior.

4.1 Design Approach / Materials & Methods

4.1.1 Architectural Paradigm

The MIRROR platform is engineered upon a distributed microservices architecture, a decision driven by the need for modularity and independent service scalability. Unlike monolithic systems where a single code failure can destabilize the entire platform, this decoupled approach ensures that individual components—such as the Auth Service and the Memory Service—operate within isolated runtime environments. This paradigm facilitates the implementation of high-availability patterns, allowing the system to maintain partial functionality even during service-specific failures.

4.1.2 The Technology Stack Selection

The selection of the technology stack was informed by a rigorous analysis of performance, security, and maintainability requirements. The backend utilizes Java 17+ with the Spring Boot 3.x framework, chosen for its robust support for enterprise-grade security and asynchronous processing.

Component	Selected Technology	Alternative Considered	Selection Rationale
Backend	Spring Boot (Java)	Django (Python)	Superior type safety and concurrency management for high load tasks.
Database	PostgreSQL + pgvector	MongoDB	Relational integrity combined with native vector search capabilities.
Message Broker	RabbitMQ	Apache Kafka	Simpler deployment and lower resource overhead for small-to-medium scale.

The comparative analysis presented above highlights the strategic preference for technologies that provide a balance between performance and architectural stability. The integration of `pgvector` within a relational database ecosystem, for instance, eliminates the synchronization overhead typically associated with managing separate vector stores, thereby optimizing the data retrieval lifecycle.

4.1.3 Development Methodology

The platform was developed following an iterative, test-driven approach to ensure that each feature satisfied the defined functional requirements. The development lifecycle integrated automated CI/CD pipelines, enabling continuous integration, automated testing, and rapid deployment. This methodology ensured that the system remained robust, with a clear audit trail of every technical change and deployment state. By utilizing containerized development environments, the project successfully mitigated the risks associated with configuration drift, ensuring that the development, testing, and production environments maintained structural

parity throughout the research lifecycle.

4.2 Codes and Standards

This section outlines the rigorous technical governance applied to the development of the MIRROR platform. Adhering to recognized industry standards is essential to ensure system security, maintainability, and interoperability.

4.2.1 Security Standards (OWASP Compliance)

The platform integrates the OWASP (Open Web Application Security Project) Top 10 as the foundational security framework. Protecting user-sensitive reflection data requires a proactive security posture:

- **Broken Access Control:** Mitigation is achieved via granular Role-Based Access Control (RBAC) enforced at the API Gateway and validated at the service level using Spring Security's method-level annotations.
- **Cryptographic Failures:** We employ BCrypt hashing with an adaptive work factor for all credential storage, ensuring that password databases are resistant to brute-force and rainbow table attacks.
- **Injection Prevention:** The usage of Spring Data JPA's parameterized queries inherently sanitizes inputs, preventing SQL injection. Furthermore, the pgvector operations utilize secure binding, insulating the vector database from malicious input patterns.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@RestController
@RequestMapping("/api/auth/admin/users")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> deleteUser(@PathVariable UUID id) {
        authService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}
```

4.2.2 Coding Standards and SOLID Principles

To manage the complexity of a microservices ecosystem, we strictly adhered to SOLID principles throughout the development lifecycle:

- **Single Responsibility Principle (SRP):** Each microservice and individual component (e.g., `EmbeddingService`, `NotificationService`) is constrained to a single, well-defined business purpose.
- **Dependency Inversion Principle (DIP):** By leveraging Spring's IoC (Inversion of Control) container, high-level modules remain decoupled from low-level implementation details, facilitating easier unit testing and future-proofing the system against infrastructure changes.

4.2.3 Data and Communication Standards

The platform relies on standardized protocols to ensure interoperability:

- **RESTful API Design:** Communication adheres to REST constraints, utilizing standard HTTP verbs and JSON as the primary payload format to ensure compatibility with diverse client platforms (Web, Android).
- **RFC-Compliant Security:** Authentication utilizes the JSON Web Token (JWT) standard (RFC 7519), ensuring that identity propagation between microservices is both secure and stateless, as required for horizontal scalability.

4.3 Constraints, Alternatives, and Tradeoffs

This section critically evaluates the architectural decisions made during the design phase. Engineering involves balancing competing requirements; this analysis documents the rationale behind selecting specific pathways while acknowledging the limitations inherent in those choices.

4.3.1 Performance Constraints and Latency Tradeoffs

The primary performance constraint centered on the real-time nature of AI inference.

- The Constraint: Providing sub-second AI responses is challenging when integrating external APIs.
- The Tradeoff: We prioritized Groq's LPU-accelerated inference over general-purpose LLM endpoints. While this choice necessitates a dependency on a specialized cloud provider, the significant reduction in token generation latency (often exceeding 500ms) was deemed essential for maintaining conversational continuity in the user interface.
- Alternative: We considered hosting open-source models (e.g., Llama 3) locally via Ollama. However, this was rejected due to the resource-intensive hardware requirements that would have hindered the platform's portability for personal-scale deployment.

4.3.2 Infrastructure and Scalability

The selection of a containerized architecture using docker-compose represents a deliberate balance between environment parity and operational complexity.

- The Constraint: Managing microservices can lead to "operational bloat" if not properly containerized.
- The Tradeoff: By utilizing Docker, we sacrificed the native performance of bare-metal installations to gain the benefit of immutable, reproducible environments. This ensures that the platform behaves identically on development workstations and production servers.
- Alternative: Kubernetes (K8s) was evaluated but deemed over-engineered for a project of this scale. The complexity of managing a K8s control plane would have consumed significant development time, detracting from core feature implementation.

4.3.3 Retrieval-Augmented Generation (RAG) vs. Model Fine-Tuning

A pivotal design decision involved how to integrate personal history into the AI's cognitive process.

- The RAG Approach: We implemented a RAG pipeline to inject context into prompts at runtime. This allows for the dynamic inclusion of a user's most relevant memories without modifying the underlying model weights.
- The Tradeoff: RAG introduces a dependency on embedding quality and retrieval relevance. If the k-NN search fails to find contextually relevant chunks, the model's response quality diminishes.
- Alternative (Fine-Tuning): Fine-tuning the LLM on user data was analyzed as a method to "bake" history into the model. This was ultimately rejected because it creates a static, expensive-to-update system that is susceptible to "catastrophic forgetting," where the model loses its general conversational capabilities while struggling to recall specific, granular reflection details.

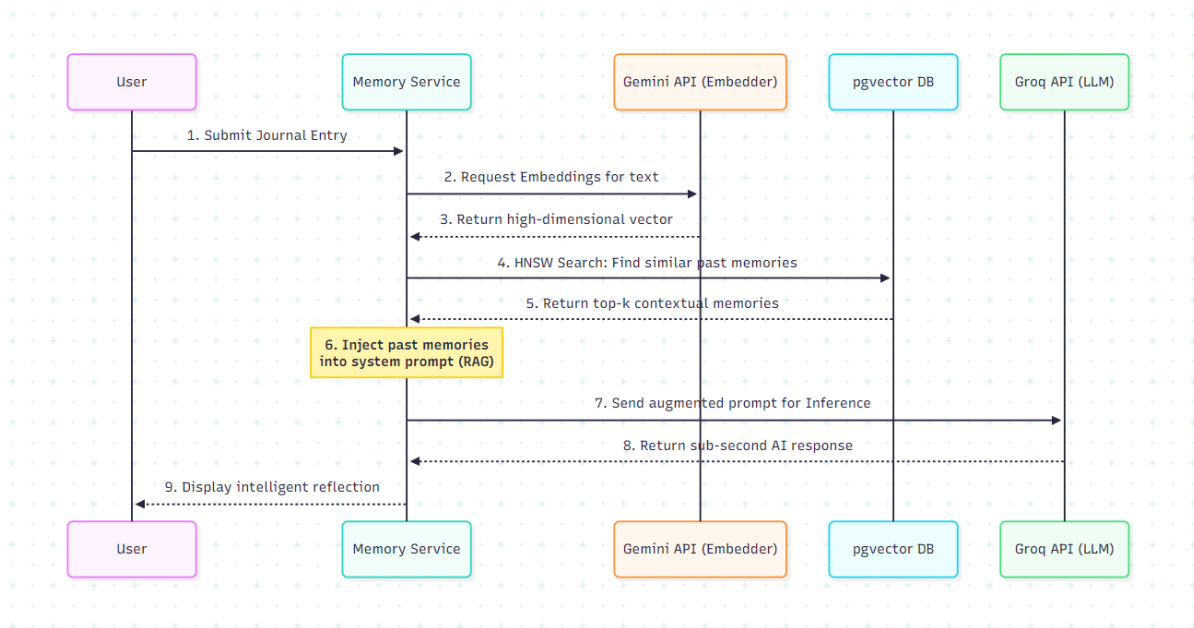


Figure 4.1: RAG-based Sequence Diagram for Cognitive Reflection.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The successful delivery of a complex, distributed research platform necessitates rigorous project management and systematic planning. This chapter documents the development lifecycle of the MIRROR platform, outlining the progression from initial architectural conception to final system validation. By documenting the planned versus actual milestones, this chapter provides a transparent view of the engineering challenges encountered and the adjustments made to ensure the platform’s technical integrity. The following sections detail the phased task allocation, the timeline of key deliverables, and a retrospective analysis of the development process, highlighting the technical rigor applied throughout the project's evolution.

5.1 Project Lifecycle Methodology

The project followed an iterative development model, prioritizing modularity and risk mitigation. This lifecycle was organized into four distinct phases: initiation, architecture design, core implementation, and validation. By adopting an iterative approach, we ensured that the AI engine—the highest-risk component—was integrated early, allowing for successive rounds of latency tuning and performance profiling against the defined requirements.

5.2 Milestone Matrix

The following matrix delineates the project timeline across the spring development cycle, mapping the planned operational phases to actual completion windows between March and June.

Milestone	Phase Focus	Planned Duration	Actual Completion
M1	Requirements & Architectural Design	3 Weeks	March 1 – March 28

M2	Backend Infrastructure & Microservices	4 Weeks	March 29 – April 25
M3	AI Engine (RAG) & Vector Integration	4 Weeks	April 26 – May 31
M4	Frontend Integration & Security Hardening	3 Weeks	June 1 – June 23
Buffer	Validation & Documentation	1 Weeks	June 24 – June 30

5.3 Task Allocation and Resource Management

Effort distribution was categorized into three primary workstreams to ensure parallel development efficiency:

- **Infrastructure Provisioning:** Included the containerization of stateful services via Docker, orchestration of the microservice network, and the configuration of the message broker for asynchronous event handling.
- **Business Logic Implementation:** Focused on the development of the Memory Service, integration of the Gemini API for embedding generation, and the enforcement of security policies at the API Gateway level.
- **Quality Assurance:** Dedicated cycles for unit testing core modules, integration testing of the RAG pipeline, and stress testing to evaluate system performance under concurrent load.

5.4 Retrospective Analysis

The development process revealed critical insights into the complexity of distributed, AI-augmented systems. The primary technical hurdle involved managing network latency and asynchronous thread execution when generating embeddings via the Gemini API and indexing them in PostgreSQL. The schedule adjustments noted in the Milestone Matrix reflect the rigorous testing required to ensure the UI remained responsive during heavy AI ingestion. This experience demonstrated that in high-performance RAG pipelines, non-blocking I/O and architectural robustness must be prioritized over feature delivery speed, as the complexity of state management dictates the stability of the entire cognitive core.

CHAPTER 6

DISSERTATION DEMONSTRATION

This chapter provides a practical demonstration of the MIRROR platform, illustrating the functional execution of the core system components developed and analyzed in the preceding chapters. By presenting representative code implementations and visual outputs from the user interface, this section validates the system's ability to fulfill its core requirements: secure authentication, intelligent memory retrieval, and real-time AI reflection. The following sections offer a technical walkthrough of key service interactions and a visual assessment of the platform's operational state, serving as the definitive verification of the research implementation.

6.1 Sample Codes

This section presents key code snippets that highlight the core technical implementations and architectural decisions of the application. These samples demonstrate how critical functionalities—such as user authentication, real-time messaging, and external service integrations—are structured and executed within the codebase. The following excerpts provide insight into the programming practices, security measures, and logic used to build the system's foundation

6.1.1 Authentication and Identity Management Phase

This section demonstrates the core security protocols and user management workflows implemented within the Auth Service.

Stateless JWT Token Generation:

Demonstrates the cryptographic signing and validation of JSON Web Tokens to ensure stateless session management.

```
package com.mirror.authservice.config.security;  
  
import com.mirror.authservice.user.model.User;
```

```

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {

    private final Key SECRET_KEY;

    public JwtUtil(@Value("${app.jwt.secret}") String secret) {
        this.SECRET_KEY =
Keys.hmacShaKeyFor(io.jsonwebtoken.io.Decoders.BASE64.decode(secret));
    }

    private final long ACCESS_TOKEN_EXPIRATION = 1000 * 60 * 15;

    public String generateAccessToken(User user) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", user.getRole().name());
        claims.put("email", user.getEmail());

        return Jwts.builder()
            .setClaims(claims)

```

```

        .setSubject(user.getUsername())
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() +
ACCESS_TOKEN_EXPIRATION))
        .signWith(SECRET_KEY)
        .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(SECRET_KEY)
            .build()
            .parseClaimsJws(token)
            .getBody();
    }

    public boolean validateToken(String token, String username) {
        final String extractedUsername = extractUsername(token);
        return (extractedUsername.equals(username) && !isTokenExpired(token));
    }

```

```

}

private boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

public String generateResetToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("purpose", "password_reset");
    claims.put("email", user.getEmail());
    claims.put("pwdHashId", Integer.toHexString(user.getPasswordHash().hashCode()));

    return Jwts.builder()
        .setClaims(claims)
        .setSubject(user.getUsername())
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 15)) // 15 mins
        .signWith(SECRET_KEY)
        .compact();
}

public boolean validateResetToken(String token, User user) {
    try {
        Claims claims = extractAllClaims(token);
        String expectedHashId = Integer.toHexString(user.getPasswordHash().hashCode());
        return "password_reset".equals(claims.get("purpose")) &&
            user.getEmail().equals(claims.get("email")) &&
            expectedHashId.equals(claims.get("pwdHashId")) &&
            !isTokenExpired(token);
    } catch (Exception e) {
        return false;
    }
}

```

```
}  
}
```

“backend\auth-service\src\main\java\com\mirror\authservice\config\security\JwtUtil.java”

Custom Spring Security JWT Interceptor:

Shows the custom filter chain that intercepts incoming HTTP requests, extracts the Bearer token, and populates the Security Context.

```
package com.mirror.authservice.config.security;  
  
import jakarta.servlet.FilterChain;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
import org.springframework.beans.factory.annotation.Autowired;  
import  
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;  
import org.springframework.security.core.authority.SimpleGrantedAuthority;  
import org.springframework.security.core.context.SecurityContextHolder;  
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;  
import org.springframework.stereotype.Component;  
import org.springframework.web.filter.OncePerRequestFilter;  
  
import java.io.IOException;  
import java.util.Collections;  
  
@Component  
public class JwtAuthenticationFilter extends OncePerRequestFilter {  
  
    @Autowired
```

```

private JwtUtil jwtUtil;

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain)
    throws ServletException, IOException {

    final String authorizationHeader = request.getHeader("Authorization");

    String username = null;
    String jwt = null;
    String role = null;

    if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
        jwt = authorizationHeader.substring(7);
        try {
            username = jwtUtil.extractUsername(jwt);
            role = jwtUtil.extractClaim(jwt, claims -> claims.get("role", String.class));
        } catch (Exception e) {
        }
    }

    if (username != null && SecurityContextHolder.getContext().getAuthentication() ==
null) {
        if (jwtUtil.validateToken(jwt, username)) {
            String springRole = (role != null && role.startsWith("ROLE_")) ? role : "ROLE_"
+ role;

            UsernamePasswordAuthenticationToken authenticationToken = new
UsernamePasswordAuthenticationToken(
                username, null, Collections.singletonList(new
SimpleGrantedAuthority(springRole)));

```

```

        authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authenticationToken);
    }
}

chain.doFilter(request, response);
}
}

```

“backend\auth-service\src\main\java\com\mirror\authservice\config\security\JwtAuthenticationFilter.java”

Role-Based Access Control (RBAC):

Illustrates the use of Spring Security’s `@PreAuthorize` annotations to restrict sensitive endpoints to administrative personnel.

```

package com.mirror.authservice.admin.controller;
import com.mirror.authservice.admin.dto.AdminCreateUserRequest;
import com.mirror.authservice.auth.service.AuthService;
import com.mirror.authservice.user.dto.UserResponse;
import com.mirror.authservice.user.dto.UserUpdateRequest;

import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import jakarta.validation.Valid;

```

```

import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/api/auth/admin/users")
@RequiredArgsConstructor
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    private final AuthService authService;

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<List<UserResponse>> getAllUsers() {
        return ResponseEntity.ok(authService.getAllUsers());
    }

    @GetMapping("/metrics")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<?> getMetrics() {
        long userCount = authService.getUserCount();
        return ResponseEntity.ok(java.util.Map.of("user_count", userCount));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<UserResponse> getUserById(@PathVariable UUID id) {
        return ResponseEntity.ok(authService.getUserById(id));
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")

```

```

    public ResponseEntity<?> createUser(@Valid @RequestBody AdminCreateUserRequest
request) {
    try {
        UserResponse created = authService.adminCreateUser(
            request.username(),
            request.email(),
            request.password(),
            request.role()
        );
        return ResponseEntity.ok(created);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@PutMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<UserResponse> updateUser(@PathVariable UUID id, @Valid
@RequestBody UserUpdateRequest request) {
    return ResponseEntity.ok(authService.updateUser(id, request));
}

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> deleteUser(@PathVariable UUID id) {
    authService.deleteUser(id);
    return ResponseEntity.noContent().build();
}
}

```

“backend\auth-service\src\main\java\com\mirror\authservice\admin\controller\AdminController.java”

Password Cryptography and Hashing:

Highlights the configuration of the BCryptPasswordEncoder bean, ensuring user credentials are mathematically protected against brute-force attacks.

```
package com.mirror.authservice.config.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity
;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;
```

```

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .httpBasic(httpBasic -> httpBasic.disable())
        .formLogin(formLogin -> formLogin.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/api/auth/signup",
                "/api/auth/login",
                "/api/auth/otp/**",
                "/api/auth/forgot-password/**",
                "/api/auth/refresh",
                "/api/auth/validate",
                "/api/auth/keepalive",
                "/error"
            )
            .permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
}

```

“backend\auth-

service\src\main\java\com\mirror\authservice\config\security\SecurityConfig.java”

Asynchronous OTP Email Delivery:

Demonstrates the use of Spring’s `@Async` annotation to offload Brevo SMTP email transmission to background threads, maintaining API responsiveness.

```
package com.mirror.authservice.config.email;

import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class EmailService {
    private final JavaMailSender mailSender;

    @Value("${app.mail.from}")
    private String fromEmail;

    @Async
    public void sendOtpEmail(String to, String otp, String username, String type) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom(fromEmail);
        message.setTo(to);

        String subject;
        String body;
```

```

if ("RESET".equalsIgnoreCase(type)) {
    subject = "Reset Your Mirror Password";
    body = "Hi " + username + ",\n\n" +
        "We received a request to reset your password. Use the verification code below
to proceed:\n\n" +
        "Your Reset Code is: " + otp + "\n" +
        "This code will expire in 5 minutes.\n\n" +
        "If you did not request this, please ignore this email.\n\n" +
        "Best regards,\n" +
        "The Mirror Security Team";
} else {
    subject = "Your Mirror Verification Code";
    body = "Hi " + username + " ✨🌟 \n\n" +
        "Your OTP code is: " + otp + "\n" +
        "This code expires in 5 minutes.\n\n" +
        "Thank you for choosing Mirror! >^__^<\n\n" +
        "Best regards,\n" +
        "The Mirror Team";
}

message.setSubject(subject);
message.setText(body);
mailSender.send(message);
}
}

```

“backend\auth-

service\src\main\java\com\mirror\authservice\config\email\EmailService.java”

6.1.2 Data Engineering and Vector Storage Phase

This section outlines the relational schema and the integration of native vector similarity search for the memory retrieval pipeline.

Relational Schema & pgvector Index Creation:

Contains the Flyway database migration script that provisions both standard relational tables and the advanced HNSW vector index.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS memories (
  id BIGSERIAL PRIMARY KEY,
  user_id VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  emotion VARCHAR(50) NOT NULL,
  sender VARCHAR(50) DEFAULT 'user',
  embedding vector(768),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS memories_embedding_idx ON memories USING hnsw
(embedding vector_cosine_ops);
```

“backend\memory-service\src\main\resources\db\migration\V1__init.sql”

Native Cosine Similarity Search Query:

Shows the native SQL query executed via Spring Data JPA, utilizing the pgvector <=> operator for high-speed k-NN memory retrieval.

```
package com.mirror.memoryservice.memory.repository;

import com.mirror.memoryservice.memory.model.Memory;
```

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

public interface MemoryRepository extends JpaRepository<Memory, Long> {

    @Modifying
    @Transactional
    @Query(value = "INSERT INTO memories (user_id, content, emotion, sender,
embedding, created_at) " +
        "VALUES (:userId, :content, :emotion, :sender, CASE WHEN :embedding IS
NULL THEN NULL ELSE CAST(:embedding AS vector) END,
CURRENT_TIMESTAMP)", nativeQuery = true)
    void saveMemoryWithEmbedding(
        @Param("userId") String userId,
        @Param("content") String content,
        @Param("emotion") String emotion,
        @Param("sender") String sender,
        @Param("embedding") String embedding
    );

    @Query(value = "SELECT id, user_id, content, emotion, sender, CAST(embedding AS
text) AS embedding, created_at FROM memories " +
        "WHERE user_id = :userId AND sender = 'user' " +
        "ORDER BY embedding <=> CAST(:embedding AS vector) " +
        "LIMIT :limit", nativeQuery = true)
    List<Memory> findSimilarMemories(
        @Param("userId") String userId,

```

```

    @Param("embedding") String embedding,
    @Param("limit") int limit
);

@Query(value = "SELECT emotion, COUNT(*) FROM memories " +
    "WHERE user_id = :userId " +
    "GROUP BY emotion", nativeQuery = true)
List<Object[]> findEmotionCounts(@Param("userId") String userId);

List<Memory> findAllByUserIdOrderByCreatedAtDescIdDesc(String userId);
@Query(value = "SELECT id, user_id, content, emotion, sender, CAST(embedding AS
text) AS embedding, created_at FROM memories " +
    "WHERE user_id = :userId " +
    "ORDER BY created_at DESC, id DESC " +
    "LIMIT :limit OFFSET :offset", nativeQuery = true)
List<Memory> findMemoriesPaginated(
    @Param("userId") String userId,
    @Param("limit") int limit,
    @Param("offset") int offset
);

@Query(value = "SELECT id, user_id, content, emotion, sender, CAST(embedding AS
text) AS embedding, created_at FROM memories " +
    "WHERE user_id = :userId AND (:cursor IS NULL OR id <
CAST(CAST(:cursor AS TEXT) AS BIGINT)) " +
    "ORDER BY id DESC " +
    "LIMIT :limit", nativeQuery = true)
List<Memory> findMemoriesKeysetPaginated(
    @Param("userId") String userId,
    @Param("limit") int limit,
    @Param("cursor") Long cursor
);

```

```

    long countByUserId(String userId);

    @Modifying
    @Transactional
    void deleteByUserId(String userId);
}

```

“backend\memory-service\src\main\java\com\mirror\memoryservice\memory\repository\MemoryRepository.java”

Entity Relationship Modeling:

Demonstrates the Object-Relational Mapping (ORM) used to translate PostgreSQL vector records into Java backend objects.

```

package com.mirror.memoryservice.memory.model;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "memories", indexes = {
    @Index(name = "idx_memory_user_id", columnList = "user_id"),
    @Index(name = "idx_memory_user_id_id_desc", columnList = "user_id, id DESC")
})
public class Memory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```

@Column(name = "user_id", nullable = false)
private String userId;

@Column(columnDefinition = "TEXT", nullable = false)
private String content;

@Column(nullable = false, length = 255)
private String emotion;

@Column(nullable = false, length = 50)
private String sender = "user";

@com.fasterxml.jackson.annotation.JsonIgnore
@Column(name = "embedding", columnDefinition = "vector(768)")
private String embedding;

@Transient
private String originalEmotionName;

private Instant createdAt = Instant.now();

public Memory() {}

public Memory(Long id, String userId, String content, String emotion, String sender,
String embedding, Instant createdAt) {
    this.id = id;
    this.userId = userId;
    this.content = content;
    this.emotion = emotion;
    this.sender = sender;
    this.embedding = embedding;
}

```

```

        this.createdAt = createdAt;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }
    public String getContent() { return content; }
    public void setContent(String content) { this.content = content; }
    public String getEmotion() { return emotion; }
    public void setEmotion(String emotion) { this.emotion = emotion; }
    public String getSender() { return sender; }
    public void setSender(String sender) { this.sender = sender; }
    public String getEmbedding() { return embedding; }
    public void setEmbedding(String embedding) { this.embedding = embedding; }
    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
    public String getOriginalEmotionName() { return originalEmotionName; }
    public void setOriginalEmotionName(String originalEmotionName) {
this.originalEmotionName = originalEmotionName; }
}

```

“backend\memory-

service\src\main\java\com\mirror\memoryservice\memory\model\Memory.java

Database Connection Configuration:

Details the specific HikariCP connection pool settings required to interface securely with the PostgreSQL instance.

```

server:
  port: 8081

```

```
spring:
  application:
    name: memory-service
  jackson:
    serialization:
      write-dates-as-timestamps: false
  flyway:
    baseline-on-migrate: true
    baseline-version: 3
    table: memory_flyway_schema_history
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    driver-class-name: org.postgresql.Driver
  hikari:
    max-lifetime: 240000
    minimum-idle: 0
    idle-timeout: 60000
  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        jdbc:
          batch_size: 50
          order_inserts: true
          order_updates: true
        show-sql: false
  data:
    redis:
```

```
  host: ${SPRING_DATA_REDIS_HOST}
  port: ${SPRING_DATA_REDIS_PORT}
  password: ${SPRING_DATA_REDIS_PASSWORD}
rabbitmq:
  host: ${SPRING_RABBITMQ_HOST}
  port: ${SPRING_RABBITMQ_PORT}
  username: ${SPRING_RABBITMQ_USERNAME}
  password: ${SPRING_RABBITMQ_PASSWORD}
  virtual-host: ${SPRING_RABBITMQ_VIRTUAL_HOST}

app:
  jwt:
    secret: ${JWT_SECRET}
  ai:
    temperature: 0.7

gemini:
  api:
    key: ${GEMINI_API_KEY:}
    url: ${GEMINI_API_URL}
  prompts:
    system: ${GEMINI_SYSTEM_PROMPT:}
  generation:
    temperature: 0.7

groq:
  api:
    key: ${GROQ_API_KEY}
    url: ${GROQ_API_URL}
  model:
    text: ${GROQ_MODEL}
```

```
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: ${ZIPKIN_URL}

---
spring:
  config:
    activate:
      on-profile: prod
```

“backend\memory-service\src\main\resources\application.yaml”

6.1.3 Artificial Intelligence and Cognitive Processing Phase

This section highlights the integration of external LLMs and asynchronous event-driven architecture to achieve real-time AI reflection.

Real-time LLM Inference:

Showcases the REST client configuration required to interface with Groq's high-speed LPUs, strictly enforcing JSON-formatted AI responses.

```
package com.mirror.memoryservice.ai.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.HttpClientErrorException;
```

```

import org.springframework.web.client.HttpServerErrorException;
import org.springframework.http.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import jakarta.annotation.PostConstruct;
import java.util.*;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.core.type.TypeReference;

@Service
public class GroqService {

    private static final Logger log = LoggerFactory.getLogger(GroqService.class);

    @Value("${groq.api.key}")
    private String apiKey;

    @Value("${groq.api.url}")
    private String apiUrl;

    @Value("${groq.model.text}")
    private String textModel;

    @Value("${app.ai.temperature:0.7}")
    private double temperature;

    private final ObjectMapper objectMapper;
    private final RestClient restClient;
    private final PromptService promptService;

```

```

public GroqService(PromptService promptService, ObjectMapper objectMapper) {
    this.promptService = promptService;
    this.objectMapper = objectMapper
        .copy()
        .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

    org.springframework.http.client.SimpleClientHttpRequestFactory factory =
        new org.springframework.http.client.SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(15_000);
    factory.setReadTimeout(30_000);
    this.restClient = RestClient.builder().requestFactory(factory).build();
}

@PostConstruct
public void validateConfig() {
    if (apiKey == null || apiKey.isBlank()) {
        throw new IllegalStateException(
            "[GroqService] GROQ_API_KEY is not set. Please configure it as an environment
variable.");
    }
    if (apiUrl == null || apiUrl.isBlank()) {
        throw new IllegalStateException(
            "[GroqService] GROQ_API_URL is not set. Please configure it as an environment
variable.");
    }
    if (textModel == null || textModel.isBlank()) {
        throw new IllegalStateException(
            "[GroqService] GROQ_MODEL is not set. Please configure it as an environment
variable.");
    }
}
}

```

```

    public Map<String, String> generateReflectionAndEmotion(String prompt, String
recentContext, String semanticContext) {
        if (prompt == null || prompt.isBlank()) {
            return Map.of(
                "reflection", "Please enter some text so I can reflect with you.",
                "emotion", "FEELINGS|NEUTRAL|#a855f7|#06b6d4"
            );
        }

        String userContent = buildUserContent(prompt, recentContext, semanticContext);
        String systemPrompt = buildSystemPrompt(recentContext, semanticContext);

        int maxRetries = 3;
        int attempt = 0;
        Exception lastException = null;

        while (attempt < maxRetries) {
            try {
                Map<String, String> result = callGroqApi(systemPrompt, userContent);

                return result;
            } catch (HttpClientErrorException e) {
                lastException = e;
                attempt++;
                int statusCode = e.getStatusCode().value();

                if (statusCode == 429) {
                    log.error("[GroqService] Rate limit hit (429). No further retries.");
                    throw new RuntimeException(

```

```

        "RATE_LIMIT_EXCEEDED: Groq API rate limit reached. Please try again
shortly.", e);
    }
    log.error("[GroqService] HTTP client error {} on attempt {}.", statusCode, attempt,
e);
    break;

} catch (HttpServerErrorException e) {
    lastException = e;
    attempt++;
    int statusCode = e.getStatusCode().value();

    if ((statusCode == 502 || statusCode == 503) && attempt < maxRetries) {
        log.warn("[GroqService] Groq overloaded ({}). Retrying attempt {}/{...}",
            statusCode, attempt, maxRetries);
        sleepWithBackoff(attempt);
        continue;
    }
    log.error("[GroqService] HTTP server error {} on attempt {}.", statusCode,
attempt, e);
    break;

} catch (MissingKeysException e) {
    lastException = e;
    attempt++;
    log.warn("[GroqService] JSON missing required keys. Retrying attempt {}/{...}",
        attempt, maxRetries);
    if (attempt < maxRetries) {
        userContent += "\n\nERROR: Your previous response was missing required
JSON fields. " +
            "You MUST include ALL of: reflection, emotion, pillar, primaryColor,
secondaryColor — " +

```

```

        "none can be null or missing.";
        continue;
    }
    break;

} catch (Exception e) {
    lastException = e;
    attempt++;
    log.error("[GroqService] Unexpected error on attempt {}. ", attempt, e);
    break;
}
}

String failureMsg = lastException != null && lastException.getMessage() != null
    ? lastException.getMessage()
    : (lastException != null ? lastException.getClass().getSimpleName() :
"Unknown Error");
    throw new RuntimeException("AI_SERVICE_ERROR: " + failureMsg, lastException);
}

@SuppressWarnings("unchecked")
private Map<String, String> callGroqApi(String systemPrompt, String userContent)
    throws Exception {

    String url = apiUri + "/chat/completions";

    Map<String, Object> systemMessage = Map.of("role", "system", "content",
systemPrompt);
    Map<String, Object> userMessage = Map.of("role", "user", "content", userContent);

    Map<String, Object> payload = new LinkedHashMap<>();
    payload.put("model", textModel);

```

```

payload.put("messages", List.of(systemMessage, userMessage));
payload.put("response_format", Map.of("type", "json_object"));
payload.put("temperature", temperature);
payload.put("max_tokens", 1024);

ResponseEntity<Map> response = restClient.post()
    .uri(url)
    .header("Authorization", "Bearer " + apiKey)
    .contentType(MediaType.APPLICATION_JSON)
    .body(payload)
    .retrieve()
    .toEntity(Map.class);

if (!response.getStatusCode().is2xxSuccessful() || response.getBody() == null) {
    throw new RuntimeException("Groq returned non-2xx or empty body: " +
response.getStatusCode());
}

Map<String, Object> body = response.getBody();
List<Map<String, Object>> choices = (List<Map<String, Object>>)
body.get("choices");

if (choices == null || choices.isEmpty()) {
    throw new RuntimeException("Groq response missing 'choices' field.");
}

Map<String, Object> message = (Map<String, Object>) choices.get(0).get("message");
if (message == null) {
    throw new RuntimeException("Groq response missing 'message' in first choice.");
}

String jsonContent = (String) message.get("content");

```

```

    if (jsonContent == null || jsonContent.isBlank()) {
        throw new RuntimeException("Groq response has blank 'content' field.");
    }

    return parseAndValidate(jsonContent);
}

private Map<String, String> parseAndValidate(String jsonContent) throws Exception {
    Map<String, String> parsed = objectMapper.readValue(
        jsonContent, new TypeReference<Map<String, String>>() {});

    String reflection = parsed.get("reflection");
    String emotion = parsed.get("emotion");

    if (reflection == null || reflection.isBlank() || emotion == null || emotion.isBlank()) {
        throw new MissingKeysException(
            "JSON has null/blank required fields. reflection=" + reflection + ", emotion=" +
emotion);
    }

    String pillar = nonBlankOrDefault(parsed.get("pillar"), "FEELINGS");
    String rawEmotion = nonBlankOrDefault(parsed.get("emotion"),
"NEUTRAL").replace("|", "-");
    String primaryColor = nonBlankOrDefault(parsed.get("primaryColor"), "#a855f7");
    String secondaryColor = nonBlankOrDefault(parsed.get("secondaryColor"),
"#06b6d4");

    parsed.put("emotion", pillar + "|" + rawEmotion + "|" + primaryColor + "|" +
secondaryColor);
    return parsed;
}

```

```

private String buildUserContent(String prompt, String recentContext, String
semanticContext) {
    return "--- RECENT CHAT HISTORY (Immediate Context) ---\n" +
        (recentContext != null && !recentContext.isBlank() ? recentContext : "No recent
chat history.") + "\n\n" +
        "--- PAST RELEVANT MEMORIES (Long-term Context) ---\n" +
        (semanticContext != null && !semanticContext.isBlank() ? semanticContext : "No
relevant past memories found.") + "\n\n" +
        "--- CURRENT USER PROMPT ---\n" +
        prompt + "\n\n" +
        "Respond ONLY with a valid JSON object containing exactly these keys: " +
        "reflection, emotion, pillar, primaryColor, secondaryColor.";
}

private String buildSystemPrompt(String recentContext, String semanticContext) {
    Map<String, Object> context = new HashMap<>();
    boolean hasPastMemories = semanticContext != null && !semanticContext.isBlank()
&& !semanticContext.equalsIgnoreCase("None");
    context.put("hasPastMemories", hasPastMemories);

    return promptService.renderSystemPrompt(context);
}

public String extractAndMergeFacts(String newMessage, String existingFacts) {
    if (newMessage == null || newMessage.isBlank()) return existingFacts;

    String systemPrompt = "You are a data extraction system. Your job is to extract
absolute, hard facts about the user from their message (e.g., their name, job, relationships,
core struggles, age). " +
        "Merge any new facts found with the existing facts. If no new facts are found,
return the existing facts exactly as they were. " +

```

"Output ONLY a raw, concise string of facts (e.g., 'User is a software engineer. User has a dog named Max.'). Do not use JSON or conversational text.";

```
String userContent = "EXISTING FACTS:\n" + (existingFacts != null ? existingFacts : "None") + "\n\nNEW MESSAGE:\n" + newMessage;
```

```
try {  
    Map<String, Object> systemMessage = Map.of("role", "system", "content",  
systemPrompt);  
    Map<String, Object> userMessage = Map.of("role", "user", "content",  
userContent);
```

```
    Map<String, Object> payload = new LinkedHashMap<>();  
    payload.put("model", textModel);  
    payload.put("messages", List.of(systemMessage, userMessage));  
    payload.put("temperature", 0.1); // Low temp for factual extraction  
    payload.put("max_tokens", 512);
```

```
String url = apiUrl + "/chat/completions";  
ResponseEntity<Map> response = restClient.post()  
    .uri(url)  
    .header("Authorization", "Bearer " + apiKey)  
    .contentType(MediaType.APPLICATION_JSON)  
    .body(payload)  
    .retrieve()  
    .toEntity(Map.class);
```

```
if (response.getStatusCode().is2xxSuccessful() && response.getBody() != null) {  
    Map<String, Object> body = response.getBody();  
    List<Map<String, Object>> choices = (List<Map<String, Object>>)  
body.get("choices");  
    if (choices != null && !choices.isEmpty()) {
```

```

        Map<String, Object> message = (Map<String, Object>)
choices.get(0).get("message");

        if (message != null) {
            return (String) message.get("content");
        }
    }
}

} catch (Exception e) {
    log.error("Failed to extract facts: {}", e.getMessage());
}

return existingFacts;
}

private String nonBlankOrDefault(String value, String defaultValue) {
    return (value != null && !value.isBlank()) ? value : defaultValue;
}

private void sleepWithBackoff(int attempt) {
    try {
        Thread.sleep(1500L * attempt);
    } catch (InterruptedException ie) {
        Thread.currentThread().interrupt();
    }
}

private static class MissingKeysException extends Exception {
    MissingKeysException(String message) { super("MISSING_KEYS: " + message); }
}
}

```

“backend\memory-service\src\main\java\com\mirror\memoryservice\ai\service\GroqService.java”

High-Dimensional Vector Embedding:

Demonstrates the integration with the Gemini API to convert user text into 768-dimensional float arrays for semantic storage.

```
package com.mirror.memoryservice.ai.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import org.springframework.http.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.*;

@Service
public class GeminiService {

    private static final Logger log = LoggerFactory.getLogger(GeminiService.class);

    @Value("${gemini.api.key}")
    private String apiKey;

    @Value("${gemini.api.url}")
    private String apiUrl;

    private final RestClient restClient;

    public GeminiService() {
```

```

    org.springframework.http.client.SimpleClientHttpRequestFactory factory = new
org.springframework.http.client.SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(10000);
    factory.setReadTimeout(15000);
    this.restClient = RestClient.builder().requestFactory(factory).build();
}

@org.springframework.cache.annotation.Cacheable(value = "embeddings", key = "#text")
public float[] getEmbedding(String text) {
    if (text == null) {
        text = "";
    }

    if (apiKey == null || apiKey.trim().isEmpty()) {
        throw new IllegalStateException("Gemini API key is not configured. Please set the
GEMINI_API_KEY environment variable.");
    }

    try {
        String url = apiUrl + "/models/gemini-embedding-001:embedContent?key=" +
apiKey;

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);

        Map<String, Object> parts = new HashMap<>();
        parts.put("text", text);

        Map<String, Object> content = new HashMap<>();
        content.put("parts", Collections.singletonList(parts));

        Map<String, Object> payload = new HashMap<>();

```

```

payload.put("model", "models/gemini-embedding-001");
payload.put("content", content);
payload.put("outputDimensionality", 768);

ResponseEntity<Map> response = restClient.post()
    .uri(url)
    .headers(h -> h.addAll(headers))
    .body(payload)
    .retrieve()
    .toEntity(Map.class);

if (response.getStatusCode().is2xxSuccessful() && response.getBody() != null) {
    Map<String, Object> body = response.getBody();

    Map<String, Object> embeddingNode = (Map<String, Object>)
body.get("embedding");
    if (embeddingNode != null) {
        List<Number> valuesList = (List<Number>) embeddingNode.get("values");
        if (valuesList != null) {

            if (valuesList.size() >= 768) {
                float[] vector = new float[768];
                for (int i = 0; i < 768; i++) {
                    vector[i] = valuesList.get(i).floatValue();
                }
                if (valuesList.size() > 768) {

                }
                return vector;
            } else {
                log.warn("Warning: Received embedding size is {}, but expected at least
768.", valuesList.size());
            }
        }
    }
}

```

```

        }
    }
}

throw new RuntimeException("Gemini Embeddings response was invalid or missing
expected payload fields.");
} catch (Exception e) {
    log.error("Failed to query Gemini Embeddings API.", e);
    throw new RuntimeException("Failed to query Gemini Embeddings API: " +
e.getMessage(), e);
}
}
}
}

```

“backend\memory-
service\src\main\java\com\mirror\memoryservice\ai\service\GeminiService.java”

RAG Pipeline Orchestration:

The core business logic that coordinates the retrieval of past memories and injects them into the current AI conversation context.

```

package com.mirror.memoryservice.memory.service;
import com.mirror.memoryservice.ai.service.GeminiService;
import com.mirror.memoryservice.ai.service.GroqService;
import com.mirror.memoryservice.admin.dto.EmotionStatDTO;
import com.mirror.memoryservice.admin.dto.AnalyticsResponseDTO;

import com.mirror.memoryservice.memory.model.Memory;
import com.mirror.memoryservice.memory.model.UserProfile;
import com.mirror.memoryservice.memory.repository.MemoryRepository;

```

```

import com.mirror.memoryservice.memory.repository.UserProfileRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.CacheEvict;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.*;
import java.time.ZonedDateTime;
import java.time.ZoneId;
import java.time.temporal.ChronoUnit;
import java.time.format.DateTimeFormatter;
import com.mirror.memoryservice.common.exception.MemoryNotFoundException;
import com.mirror.memoryservice.common.exception.MemoryProcessingException;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.mirror.memoryservice.ai.event.ReflectionSaveEvent;
import com.mirror.memoryservice.config.messaging.RabbitMQConfig;

@Service
public class MemoryServiceImpl implements MemoryService {

    private static final Logger log = LoggerFactory.getLogger(MemoryServiceImpl.class);
    private static final String CACHE_EMOTION_ANALYTICS = "emotionAnalytics";

    private final MemoryRepository repository;
    private final GeminiService geminiService;
    private final GroqService groqService;
    private final RabbitTemplate rabbitTemplate;
    private final ObjectMapper objectMapper;

```

```

private final UserProfileRepository userProfileRepository;

public MemoryServiceImpl(MemoryRepository repository, GeminiService geminiService,
GroqService groqService, RabbitTemplate rabbitTemplate, ObjectMapper objectMapper,
UserProfileRepository userProfileRepository) {
    this.repository = repository;
    this.geminiService = geminiService;
    this.groqService = groqService;
    this.rabbitTemplate = rabbitTemplate;
    this.objectMapper = objectMapper;
    this.userProfileRepository = userProfileRepository;
}

@Override
@Transactional
@CacheEvict(value = CACHE_EMOTION_ANALYTICS, key = "#userId")
public String saveMemory(String userId, String content, String emotion, String sender,
float[] embedding) {
    try {
        String embeddingStr = formatVectorForSql(embedding);
        repository.saveMemoryWithEmbedding(userId, content, emotion, sender,
embeddingStr);
        return "Memory successfully cataloged and indexed semantically.";
    } catch (Exception e) {
        log.error("Error saving memory to Postgres: {}", e.getMessage(), e);
        throw new MemoryProcessingException("Database error saving memory: " +
e.getMessage(), e);
    }
}

@Override
public List<Memory> getSimilarMemories(String userId, String prompt, int limit) {

```

```

    try {
        float[] queryEmbedding = geminiService.getEmbedding(prompt);
        String embeddingStr = formatVectorForSql(queryEmbedding);
        return repository.findSimilarMemories(userId, embeddingStr, limit);
    } catch (Exception e) {
        log.error("Error searching similar memories: {}", e.getMessage(), e);
        return Collections.emptyList();
    }
}

private static class EmotionGroupingResult {
    List<com.mirror.memoryservice.admin.dto.EmotionStatDTO> groupedStats;
    Map<String, String> mapping;
    long total;
}

private int[] parseColorToRgb(String color) {
    if (color == null) return new int[]{0,0,0};
    if (color.startsWith("#") && color.length() >= 7) {
        try {
            return new int[]{
                Integer.valueOf(color.substring(1, 3), 16),
                Integer.valueOf(color.substring(3, 5), 16),
                Integer.valueOf(color.substring(5, 7), 16)
            };
        } catch (Exception e) {}
    }
    return new int[]{0,0,0};
}

private double colorDistance(String c1, String c2) {
    int[] rgb1 = parseColorToRgb(c1);

```

```

    int[] rgb2 = parseColorToRgb(c2);
    return Math.sqrt(Math.pow(rgb1[0] - rgb2[0], 2) + Math.pow(rgb1[1] - rgb2[1], 2) +
Math.pow(rgb1[2] - rgb2[2], 2));
}

private EmotionGroupingResult getGroupedEmotions(String userId) {
    List<Object[]> counts = repository.findEmotionCounts(userId);
    List<com.mirror.memoryservice.admin.dto.EmotionStatDTO> rawStats = new
ArrayList<>();
    long total = 0;

    for (Object[] row : counts) {
        if (row.length == 2 && row[0] != null) {
            String rawKey = row[0].toString();
            Long count = ((Number) row[1]).longValue();
            com.mirror.memoryservice.admin.dto.EmotionStatDTO stat =
parseEmotionTag(rawKey);
            stat.setCount(count);
            rawStats.add(stat);
            total += count;
        }
    }

    rawStats.sort((a, b) -> Long.compare(b.getCount(), a.getCount()));

    List<com.mirror.memoryservice.admin.dto.EmotionStatDTO> groupedStats = new
ArrayList<>();
    Map<String, String> mapping = new HashMap<>();
    int COLOR_THRESHOLD = 60;

    for (com.mirror.memoryservice.admin.dto.EmotionStatDTO stat : rawStats) {
        boolean merged = false;

```

```

        for (com.mirror.memoryservice.admin.dto.EmotionStatDTO group : groupedStats) {
            boolean nameMatch = stat.getName() != null && group.getName() != null &&
                stat.getName().trim().equalsIgnoreCase(group.getName().trim());
            boolean colorMatch = colorDistance(stat.getPrimaryColor(),
group.getPrimaryColor()) < COLOR_THRESHOLD;

            if (nameMatch || colorMatch) {
                mapping.put(stat.getKey(), group.getKey());
                group.setCount(group.getCount() + stat.getCount());
                merged = true;
                break;
            }
        }
        if (!merged) {
            groupedStats.add(stat);
            mapping.put(stat.getKey(), stat.getKey());
        }
    }

    EmotionGroupingResult result = new EmotionGroupingResult();
    result.groupedStats = groupedStats;
    result.mapping = mapping;
    result.total = total;
    return result;
}

@Override
@Transactional(readOnly = true)
@Cacheable(value = CACHE_EMOTION_ANALYTICS, key = "#userId")
public com.mirror.memoryservice.admin.dto.AnalyticsResponseDTO
getEmotionalAnalytics(String userId) {

```

```

        com.mirror.memoryservice.admin.dto.AnalyticsResponseDTO response = new
com.mirror.memoryservice.admin.dto.AnalyticsResponseDTO();

        EmotionGroupingResult grouping = getGroupedEmotions(userId);
        List<com.mirror.memoryservice.admin.dto.EmotionStatDTO> stats =
grouping.groupedStats;
        long total = grouping.total;

        String dominantEmotion = "CALM";
        long maxCount = 0;

        for (com.mirror.memoryservice.admin.dto.EmotionStatDTO stat : stats) {
            if (total > 0) {
                stat.setPercentage(((int) Math.round(((double) stat.getCount() / total) * 100)));
            } else {
                stat.setPercentage(0);
            }
            if (stat.getCount() > maxCount) {
                maxCount = stat.getCount();
                dominantEmotion = stat.getKey();
            }
        }

        stats.sort((a, b) -> Long.compare(b.getCount(), a.getCount()));

        int activeStreak = total == 0 ? 0 : Math.max(1, Math.min(12, (int) (total / 4) + 1));

        String auraGradient = generateAuraGradient(stats, total);

        response.setTotalMemories(total);
        response.setDominantEmotion(dominantEmotion);
        response.setActiveStreak(activeStreak);

```

```

        response.setEmotionStats(stats);
        response.setAuraGradient(auraGradient);

        return response;
    }

    private com.mirror.memoryservice.admin.dto.EmotionStatDTO parseEmotionTag(String
rawTag) {
        com.mirror.memoryservice.admin.dto.EmotionStatDTO dto = new
com.mirror.memoryservice.admin.dto.EmotionStatDTO();
        dto.setKey(rawTag);

        if (rawTag == null || rawTag.isEmpty()) {
            dto.setPillar("FEELINGS");
            dto.setName("Neutral");
            dto.setPrimaryColor("#7928ca");
            dto.setSecondaryColor("#ff0080");
            return dto;
        }

        String[] parts = rawTag.split("\\|");

        if (parts.length >= 4) {
            dto.setPillar(parts[0]);
            dto.setName(parts[1]);
            dto.setPrimaryColor(parts[2]);
            dto.setSecondaryColor(parts[3]);
        } else {
            dto.setPillar("FEELINGS");
            dto.setName(parts.length > 0 ? parts[0] : "Neutral");
            dto.setPrimaryColor("#7928ca");
            dto.setSecondaryColor("#ff0080");
        }
    }

```

```

    }

    return dto;
}

private String
generateAuraGradient(List<com.mirror.memoryservice.admin.dto.EmotionStatDTO> stats,
long total) {
    if (total == 0 || stats.isEmpty()) {
        return "transparent";
    }

    int currentPercent = 0;
    List<String> gradientParts = new ArrayList<>();

    for (com.mirror.memoryservice.admin.dto.EmotionStatDTO stat : stats) {
        if (stat.getPercentage() > 0) {
            int nextPercent = currentPercent + stat.getPercentage();
            gradientParts.add(String.format("%s %d%% %d%%", stat.getPrimaryColor(),
currentPercent, nextPercent));
            currentPercent = nextPercent;
        }
    }

    if (currentPercent < 100 && !gradientParts.isEmpty()) {
        gradientParts.add(String.format("%s %d%% 100%%", stats.get(0).getPrimaryColor(),
currentPercent));
    }

    return "conic-gradient(" + String.join(" ", gradientParts) + ")";
}

```

```

@Override
public Map<String, String> generateReflection(String userId, String prompt) {
    List<Memory> pastMemories = getSimilarMemories(userId, prompt, 5);
    StringBuilder semanticContextBuilder = new StringBuilder();
    if (!pastMemories.isEmpty()) {
        for (int i = 0; i < pastMemories.size(); i++) {
            Memory m = pastMemories.get(i);
            String friendlyEmotionName = parseEmotionTag(m.getEmotion()).getName();
            semanticContextBuilder.append(String.format("Memory %d: %s (Emotion tagged:
%s)\n", i + 1, m.getContent(), friendlyEmotionName));
        }
    } else {
        semanticContextBuilder.append("No relevant past memories found.");
    }

    List<Memory> recentMemoriesRaw = repository.findMemoriesKeysetPaginated(userId,
40, null);
    List<Memory> recentMemoriesFiltered = new ArrayList<>();
    ZonedDateTime cutoffTime = ZonedDateTime.now(ZoneId.systemDefault()).minus(24,
ChronoUnit.HOURS);

    for (int i = 0; i < recentMemoriesRaw.size(); i++) {
        Memory m = recentMemoriesRaw.get(i);
        if (i < 10) {
            recentMemoriesFiltered.add(m);
        } else {
            ZonedDateTime msgTime = m.getCreatedAt().atZone(ZoneId.systemDefault());
            if (msgTime.isAfter(cutoffTime)) {
                recentMemoriesFiltered.add(m);
            } else {
                break;
            }
        }
    }
}

```

```

    }
}

List<Memory> recentMemories = new ArrayList<>(recentMemoriesFiltered);
Collections.reverse(recentMemories);

ZonedDateTime now = ZonedDateTime.now(ZoneId.systemDefault());
String timeElapsedStr = "This is the first interaction.";
if (!recentMemoriesRaw.isEmpty()) {
    ZonedDateTime lastTime =
recentMemoriesRaw.get(0).getCreatedAt().atZone(ZoneId.systemDefault());
    long hours = ChronoUnit.HOURS.between(lastTime, now);
    if (hours > 24) {
        timeElapsedStr = (hours / 24) + " days ago.";
    } else if (hours > 0) {
        timeElapsedStr = hours + " hours ago.";
    } else {
        long minutes = ChronoUnit.MINUTES.between(lastTime, now);
        timeElapsedStr = minutes + " minutes ago.";
    }
}

com.mirror.memoryservice.admin.dto.AnalyticsResponseDTO analytics =
getEmotionalAnalytics(userId);

UserProfile profile = userProfileRepository.findById(userId).orElse(null);

StringBuilder recentContextBuilder = new StringBuilder();
recentContextBuilder.append("--- SYSTEM REALITY CONTEXT ---\n");
recentContextBuilder.append("Current Time:
").append(now.format(DateTimeFormatter.ofPattern("EEEE, hh:mm a"))).append("\n");
recentContextBuilder.append("Time since last interaction:
").append(timeElapsedStr).append("\n");

```

```

        recentContextBuilder.append("User's Recent Dominant Emotion:
").append(analytics.getDominantEmotion() != null ? analytics.getDominantEmotion() :
"UNKNOWN").append("\n");

        if (profile != null && profile.getCoreFacts() != null &&
!profile.getCoreFacts().isBlank()) {
            recentContextBuilder.append("Known User Facts:
").append(profile.getCoreFacts()).append("\n");
        }
        recentContextBuilder.append("\n");

        if (!recentMemories.isEmpty()) {
            for (Memory m : recentMemories) {
                String senderStr = m.getSender().equalsIgnoreCase("mirror") ? "Mirror" : "User";
                recentContextBuilder.append(String.format("%s: %s\n", senderStr,
m.getContent()));
            }
        } else {
            recentContextBuilder.append("No recent chat history.");
        }

        Map<String, String> aiResponse = groqService.generateReflectionAndEmotion(prompt,
recentContextBuilder.toString(), semanticContextBuilder.toString());

        try {
            String detectedEmotion = aiResponse.getDefault("emotion", "NEUTRAL");
            String reflectionText = aiResponse.getDefault("reflection", "");

            ReflectionSaveEvent event = new ReflectionSaveEvent(userId, prompt,
reflectionText, detectedEmotion);

            String message = objectMapper.writeValueAsString(event);
            rabbitTemplate.convertAndSend(RabbitMQConfig.REFLECTION_SAVE_QUEUE,
message);

```

```

    } catch (Exception e) {
        log.error("Could not auto-save RAG interaction to RabbitMQ: {}", e.getMessage());
    }

    return aiResponse;
}

private List<Memory> mapMemoriesWithGroupedEmotions(String userId,
List<Memory> memories) {
    Map<String, String> mapping = getGroupedEmotions(userId).mapping;
    List<Memory> mapped = new ArrayList<>();
    for (Memory m : memories) {
        Memory newMem = new Memory(m.getId(), m.getUserId(), m.getContent(),
m.getEmotion(), m.getSender(), m.getEmbedding(), m.getCreatedAt());
        newMem.setEmotion(mapping.getDefault(m.getEmotion(), m.getEmotion()));
        newMem.setOriginalEmotionName(parseEmotionTag(m.getEmotion()).getName());
        mapped.add(newMem);
    }
    return mapped;
}

@Override
@Transactional(readonly = true)
public List<Memory> getAllMemories(String userId) {
    List<Memory> memories =
repository.findAllByUserIdOrderByCreatedAtDescIdDesc(userId);
    return mapMemoriesWithGroupedEmotions(userId, memories);
}

@Override
@Transactional(readonly = true)

```

```

public Map<String, Object> getMemoriesPaginated(String userId, Long cursor, int size) {
    List<Memory> memories = repository.findMemoriesKeysetPaginated(userId, size,
cursor);

    memories = mapMemoriesWithGroupedEmotions(userId, memories);

    Long total = null;
    if (cursor == null) {
        total = repository.countByUserId(userId);
    }

    boolean hasMore = memories.size() == size;
    Long nextCursor = memories.isEmpty() ? null : memories.get(memories.size() -
1).getId();

    Map<String, Object> result = new HashMap<>();
    result.put("messages", memories);
    if (total != null) {
        result.put("total", total);
    }
    result.put("hasMore", hasMore);
    result.put("nextCursor", nextCursor);
    return result;
}

@Override
@Transactional(readOnly = true)
public long getMemoryCount(String userId) {
    return repository.countByUserId(userId);
}

@Override
@Transactional

```

```

public void deleteMemory(Long id) {
    repository.deleteById(id);
}

@Override
@Transactional
@CacheEvict(value = CACHE_EMOTION_ANALYTICS, key = "#userId")
public void updateMemory(Long id, String userId, String content, String emotion) {
    Memory memory = repository.findById(id).orElseThrow(() -> new
MemoryNotFoundException("Memory " + id + " not found"));

    if (!memory.getContent().equals(content)) {
        float[] newEmbedding = geminiService.getEmbedding(content);
        String embeddingStr = "[" + java.util.stream.IntStream.range(0,
newEmbedding.length)
            .mapToObj(i -> String.valueOf(newEmbedding[i]))
            .collect(java.util.stream.Collectors.joining(", ")) + "]";
        memory.setEmbedding(embeddingStr);
    }

    memory.setUserId(userId);
    memory.setContent(content);
    memory.setEmotion(emotion);
    repository.save(memory);
}

@Override
@Transactional(readOnly = true)
public List<Memory> getAllMemoriesAdmin() {
    return repository.findAll();
}

```

```

@Override
@Transactional
@CacheEvict(value = CACHE_EMOTION_ANALYTICS, key = "#userId")
public void deleteAllMemoriesForUser(String userId) {
    log.info("Deleting all memories for user: {}", userId);
    repository.deleteByUserId(userId);
}

private String formatVectorForSql(float[] vector) {
    if (vector == null || vector.length == 0) {
        return null;
    }
    return java.util.Arrays.toString(vector);
}
}

```

“backend\memory-
service\src\main\java\com\mirror\memoryservice\memory\service\MemoryServiceImpl.java”

Background Event Processing:

Proves the decoupling of the microservices by showing how the Memory Service listens to RabbitMQ queues to process heavy AI workloads asynchronously.

```

package com.mirror.memoryservice.config.messaging;
import com.mirror.memoryservice.ai.event.ReflectionSaveEvent;
import com.mirror.memoryservice.ai.service.GeminiService;
import com.mirror.memoryservice.ai.service.GroqService;
import com.mirror.memoryservice.memory.service.MemoryService;

import com.fasterxml.jackson.databind.ObjectMapper;

```

```

import com.mirror.memoryservice.config.messaging.RabbitMQConfig;
import com.mirror.memoryservice.memory.event.MemorySaveEvent;
import com.mirror.memoryservice.memory.model.UserProfile;
import com.mirror.memoryservice.memory.repository.UserProfileRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

import java.util.Map;

@Service
public class MemoryEventListener {

    private static final Logger log = LoggerFactory.getLogger(MemoryEventListener.class);
    private final MemoryService memoryService;
    private final GeminiService geminiService;
    private final GroqService groqService;
    private final ObjectMapper objectMapper;
    private final UserProfileRepository userProfileRepository;

    public MemoryEventListener(MemoryService memoryService, GeminiService
geminiService, GroqService groqService, ObjectMapper objectMapper,
UserProfileRepository userProfileRepository) {
        this.memoryService = memoryService;
        this.geminiService = geminiService;
        this.groqService = groqService;
        this.objectMapper = objectMapper;
        this.userProfileRepository = userProfileRepository;
    }

    @RabbitListener(queues = RabbitMQConfig.MEMORY_SAVE_QUEUE)

```

```

public void handleMemorySaveEvent(String message) {
    try {
        MemorySaveEvent event = objectMapper.readValue(message,
MemorySaveEvent.class);

        float[] embedding = geminiService.getEmbedding(event.content());
        Map<String, String> sentiment =
groqService.generateReflectionAndEmotion(event.content(), null, null);
        String emotion = sentiment.getOrDefault("emotion", "NEUTRAL");

        memoryService.saveMemory(event.userId(), event.content(), emotion, "user",
embedding);

        UserProfile profile = userProfileRepository.findById(event.userId()).orElse(new
UserProfile(event.userId(), null));
        String updatedFacts = groqService.extractAndMergeFacts(event.content(),
profile.getCoreFacts());
        profile.setCoreFacts(updatedFacts);
        userProfileRepository.save(profile);

    } catch (Exception e) {
        log.error("Failed to process memory save event", e);
    }
}

@RabbitListener(queues = RabbitMQConfig.REFLECTION_SAVE_QUEUE)
public void handleReflectionSaveEvent(String message) {
    try {
        com.mirror.memoryservice.ai.event.ReflectionSaveEvent event =
objectMapper.readValue(message,
com.mirror.memoryservice.ai.event.ReflectionSaveEvent.class);

        float[] promptEmbedding = geminiService.getEmbedding(event.prompt());

```

```

        memoryService.saveMemory(event.userId(), event.prompt(), event.emotion(), "user",
promptEmbedding);

        memoryService.saveMemory(event.userId(), event.reflection(), event.emotion(),
"mirror", null);

    } catch (Exception e) {
        log.error("Failed to process background reflection save event", e);
    }
}

@RabbitListener(queues = RabbitMQConfig.USER_DELETE_QUEUE)
public void handleUserDeleteEvent(String username) {
    try {
        log.info("Received user deletion event for username: {}", username);
        memoryService.deleteAllMemoriesForUser(username);
    } catch (Exception e) {
        log.error("Failed to process user delete event", e);
    }
}
}

```

“backend\memory-
service\src\main\java\com\mirror\memoryservice\config\messaging\MemoryEventListener.java”

Automated Fact Extraction:

Illustrates how dynamic system prompts are engineered to force the AI to extract hard facts from user reflections.

```

package com.mirror.memoryservice.ai.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.http.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import jakarta.annotation.PostConstruct;
import java.util.*;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.core.type.TypeReference;

@Service
public class GroqService {

    private static final Logger log = LoggerFactory.getLogger(GroqService.class);

    @Value("${groq.api.key}")
    private String apiKey;

    @Value("${groq.api.url}")
    private String apiUrl;

    @Value("${groq.model.text}")
    private String textModel;

    @Value("${app.ai.temperature:0.7}")

```

```

private double temperature;

private final ObjectMapper objectMapper;
private final RestClient restClient;
private final PromptService promptService;

public GroqService(PromptService promptService, ObjectMapper objectMapper) {
    this.promptService = promptService;
    this.objectMapper = objectMapper
        .copy()
        .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

    org.springframework.http.client.SimpleClientHttpRequestFactory factory =
        new org.springframework.http.client.SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(15_000);
    factory.setReadTimeout(30_000);
    this.restClient = RestClient.builder().requestFactory(factory).build();
}

@PostConstruct
public void validateConfig() {
    if (apiKey == null || apiKey.isBlank()) {
        throw new IllegalStateException(
            "[GroqService] GROQ_API_KEY is not set. Please configure it as an environment
variable.");
    }
    if (apiUrl == null || apiUrl.isBlank()) {
        throw new IllegalStateException(
            "[GroqService] GROQ_API_URL is not set. Please configure it as an environment
variable.");
    }
    if (textModel == null || textModel.isBlank()) {

```

```

        throw new IllegalStateException(
            "[GroqService] GROQ_MODEL is not set. Please configure it as an environment
variable.");
    }

}

public Map<String, String> generateReflectionAndEmotion(String prompt, String
recentContext, String semanticContext) {
    if (prompt == null || prompt.isBlank()) {
        return Map.of(
            "reflection", "Please enter some text so I can reflect with you.",
            "emotion", "FEELINGS|NEUTRAL|#a855f7|#06b6d4"
        );
    }

    String userContent = buildUserContent(prompt, recentContext, semanticContext);
    String systemPrompt = buildSystemPrompt(recentContext, semanticContext);

    int maxRetries = 3;
    int attempt = 0;
    Exception lastException = null;

    while (attempt < maxRetries) {
        try {
            Map<String, String> result = callGroqApi(systemPrompt, userContent);

            return result;

        } catch (HttpClientErrorException e) {
            lastException = e;
            attempt++;
        }
    }
}

```

```

        int statusCode = e.getStatusCode().value();

        if (statusCode == 429) {
            log.error("[GroqService] Rate limit hit (429). No further retries.");
            throw new RuntimeException(
                "RATE_LIMIT_EXCEEDED: Groq API rate limit reached. Please try again
shortly.", e);
        }
        log.error("[GroqService] HTTP client error {} on attempt {}.", statusCode, attempt,
e);

        break;

    } catch (HttpServerErrorException e) {
        lastException = e;
        attempt++;
        int statusCode = e.getStatusCode().value();

        if ((statusCode == 502 || statusCode == 503) && attempt < maxRetries) {
            log.warn("[GroqService] Groq overloaded ({}). Retrying attempt {}/{}/...",
                statusCode, attempt, maxRetries);
            sleepWithBackoff(attempt);
            continue;
        }
        log.error("[GroqService] HTTP server error {} on attempt {}.", statusCode,
attempt, e);

        break;

    } catch (MissingKeysException e) {
        lastException = e;
        attempt++;
        log.warn("[GroqService] JSON missing required keys. Retrying attempt {}/{}/...",
            attempt, maxRetries);
    }
}

```

```

        if (attempt < maxRetries) {
            userContent += "\n\nERROR: Your previous response was missing required
JSON fields. " +
                "You MUST include ALL of: reflection, emotion, pillar, primaryColor,
secondaryColor — " +
                "none can be null or missing.";
            continue;
        }
        break;

    } catch (Exception e) {
        lastException = e;
        attempt++;
        log.error("[GroqService] Unexpected error on attempt {}", attempt, e);
        break;
    }
}

String failureMsg = lastException != null && lastException.getMessage() != null
    ? lastException.getMessage()
    : (lastException != null ? lastException.getClass().getSimpleName() :
"Unknown Error");
    throw new RuntimeException("AI_SERVICE_ERROR: " + failureMsg, lastException);
}

@SuppressWarnings("unchecked")
private Map<String, String> callGroqApi(String systemPrompt, String userContent)
    throws Exception {

    String url = apiUrl + "/chat/completions";

```

```

    Map<String, Object> systemMessage = Map.of("role", "system", "content",
systemPrompt);

    Map<String, Object> userMessage = Map.of("role", "user", "content", userContent);

    Map<String, Object> payload = new LinkedHashMap<>();
    payload.put("model", textModel);
    payload.put("messages", List.of(systemMessage, userMessage));
    payload.put("response_format", Map.of("type", "json_object"));
    payload.put("temperature", temperature);
    payload.put("max_tokens", 1024);

    ResponseEntity<Map> response = restClient.post()
        .uri(url)
        .header("Authorization", "Bearer " + apiKey)
        .contentType(MediaType.APPLICATION_JSON)
        .body(payload)
        .retrieve()
        .toEntity(Map.class);

    if (!response.getStatusCode().is2xxSuccessful() || response.getBody() == null) {
        throw new RuntimeException("Groq returned non-2xx or empty body: " +
response.getStatusCode());
    }

    Map<String, Object> body = response.getBody();
    List<Map<String, Object>> choices = (List<Map<String, Object>>)
body.get("choices");

    if (choices == null || choices.isEmpty()) {
        throw new RuntimeException("Groq response missing 'choices' field.");
    }

```

```

Map<String, Object> message = (Map<String, Object>) choices.get(0).get("message");
if (message == null) {
    throw new RuntimeException("Groq response missing 'message' in first choice.");
}

String jsonContent = (String) message.get("content");
if (jsonContent == null || jsonContent.isBlank()) {
    throw new RuntimeException("Groq response has blank 'content' field.");
}

return parseAndValidate(jsonContent);
}

private Map<String, String> parseAndValidate(String jsonContent) throws Exception {
    Map<String, String> parsed = objectMapper.readValue(
        jsonContent, new TypeReference<Map<String, String>>() {});

    String reflection = parsed.get("reflection");
    String emotion = parsed.get("emotion");

    if (reflection == null || reflection.isBlank() || emotion == null || emotion.isBlank()) {
        throw new MissingKeysException(
            "JSON has null/blank required fields. reflection=" + reflection + ", emotion=" +
emotion);
    }

    String pillar = nonBlankOrDefault(parsed.get("pillar"), "FEELINGS");
    String rawEmotion = nonBlankOrDefault(parsed.get("emotion"),
"NEUTRAL").replace("|", "-");

    String primaryColor = nonBlankOrDefault(parsed.get("primaryColor"), "#a855f7");
    String secondaryColor = nonBlankOrDefault(parsed.get("secondaryColor"),
"#06b6d4");

```

```

        parsed.put("emotion", pillar + "|" + rawEmotion + "|" + primaryColor + "|" +
secondaryColor);
        return parsed;
    }

    private String buildUserContent(String prompt, String recentContext, String
semanticContext) {
        return "--- RECENT CHAT HISTORY (Immediate Context) ---\n" +
            (recentContext != null && !recentContext.isBlank() ? recentContext : "No recent
chat history.") + "\n\n" +
            "--- PAST RELEVANT MEMORIES (Long-term Context) ---\n" +
            (semanticContext != null && !semanticContext.isBlank() ? semanticContext : "No
relevant past memories found.") + "\n\n" +
            "--- CURRENT USER PROMPT ---\n" +
            prompt + "\n\n" +
            "Respond ONLY with a valid JSON object containing exactly these keys: " +
            "reflection, emotion, pillar, primaryColor, secondaryColor.";
    }

    private String buildSystemPrompt(String recentContext, String semanticContext) {
        Map<String, Object> context = new HashMap<>();
        boolean hasPastMemories = semanticContext != null && !semanticContext.isBlank()
&& !semanticContext.equalsIgnoreCase("None");
        context.put("hasPastMemories", hasPastMemories);

        return promptService.renderSystemPrompt(context);
    }

    public String extractAndMergeFacts(String newMessage, String existingFacts) {
        if (newMessage == null || newMessage.isBlank()) return existingFacts;
    }

```

```

String systemPrompt = "You are a data extraction system. Your job is to extract
absolute, hard facts about the user from their message (e.g., their name, job, relationships,
core struggles, age). " +
    "Merge any new facts found with the existing facts. If no new facts are found,
return the existing facts exactly as they were. " +
    "Output ONLY a raw, concise string of facts (e.g., 'User is a software engineer.
User has a dog named Max.'). Do not use JSON or conversational text.";

String userContent = "EXISTING FACTS:\n" + (existingFacts != null ? existingFacts :
"None") + "\n\nNEW MESSAGE:\n" + newMessage;

try {
    Map<String, Object> systemMessage = Map.of("role", "system", "content",
systemPrompt);
    Map<String, Object> userMessage = Map.of("role", "user", "content",
userContent);

    Map<String, Object> payload = new LinkedHashMap<>();
    payload.put("model", textModel);
    payload.put("messages", List.of(systemMessage, userMessage));
    payload.put("temperature", 0.1); // Low temp for factual extraction
    payload.put("max_tokens", 512);

    String url = apiUrl + "/chat/completions";
    ResponseEntity<Map> response = restClient.post()
        .uri(url)
        .header("Authorization", "Bearer " + apiKey)
        .contentType(MediaType.APPLICATION_JSON)
        .body(payload)
        .retrieve()
        .toEntity(Map.class);

```

```

        if (response.getStatusCode().is2xxSuccessful() && response.getBody() != null) {
            Map<String, Object> body = response.getBody();
            List<Map<String, Object>> choices = (List<Map<String, Object>>)
body.get("choices");
            if (choices != null && !choices.isEmpty()) {
                Map<String, Object> message = (Map<String, Object>)
choices.get(0).get("message");
                if (message != null) {
                    return (String) message.get("content");
                }
            }
        } catch (Exception e) {
            log.error("Failed to extract facts: {}", e.getMessage());
        }
        return existingFacts;
    }

    private String nonBlankOrDefault(String value, String defaultValue) {
        return (value != null && !value.isBlank()) ? value : defaultValue;
    }

    private void sleepWithBackoff(int attempt) {
        try {
            Thread.sleep(1500L * attempt);
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
        }
    }

    private static class MissingKeysException extends Exception {
        MissingKeysException(String message) { super("MISSING_KEYS: " + message); }
    }

```

```
}  
}
```

“backend\memory-
service\src\main\java\com\mirror\memoryservice\ai\service\GroqService.java”

6.1.4 Cloud Infrastructure and Resilience Phase

This section provides the configuration details for the API Gateway, focusing on traffic routing, observability, and system fault tolerance.

Dynamic Route Predicates & CORS:

Shows how incoming traffic is dynamically routed to the correct backend microservice while safely managing Cross-Origin requests from the Angular frontend.

```
globalcors:  
  add-to-simple-url-handler-mapping: true  
  cors-configurations:  
    '["/*"]':  
      allowedOrigins:  
        - "http://localhost:4200"  
        - "http://localhost:8100"  
        - "http://192.168.1.101:8100"  
        - "http://localhost"  
        - "capacitor://localhost"  
        - "https://projectmirror.tech"  
      allowedMethods:  
        - GET  
        - POST  
        - PUT  
        - DELETE
```

```

- OPTIONS
  allowedHeaders: "*"
  allowCredentials: true

routes:
- id: auth-service-route
  uri: http://localhost:8080
  predicates:
    - Path=/api/auth/**
  filters:
    - name: RequestRateLimiter
      args:
        redis-rate-limiter.replenishRate: 5
        redis-rate-limiter.burstCapacity: 10
        redis-rate-limiter.requestedTokens: 1
        key-resolver: "#{@userKeyResolver}"
    - name: CircuitBreaker
      args:
        name: defaultCircuitBreaker

- id: memory-service-route
  uri: http://localhost:8081
  predicates:
    - Path=/api/memory/**,/api/admin/memory/**
  filters:
    - AuthenticationFilter

```

Distributed Tracing Configuration:

Highlights the Zipkin configuration used to append unique trace IDs to requests for end-to-

end latency monitoring.

```
tracing:
  sampling:
    probability: 1.0
zipkin:
  tracing:
    endpoint: ${ZIPKIN_URL}
```

Redis Token-Bucket Rate Limiting:

Demonstrates how the system uses Redis to limit API requests per user, protecting the backend from Denial of Service (DoS) attacks.

```
- name: RequestRateLimiter
  args:
    redis-rate-limiter.replenishRate: 5
    redis-rate-limiter.burstCapacity: 10
    redis-rate-limiter.requestedTokens: 1
    key-resolver: "#{@userKeyResolver}"
```

Fault Tolerance via Circuit Breaking:

Shows the Resilience4j configuration that prevents cascading failures by tripping the circuit if an external AI API becomes unresponsive.

```
resilience4j:
  timelimiter:
    instances:
      defaultCircuitBreaker:
        timeoutDuration: 60s
  circuitbreaker:
```

```
instances:
  defaultCircuitBreaker:
    slidingWindowSize: 10
    failureRateThreshold: 50
    waitDurationInOpenState: 10s
    permittedNumberOfCallsInHalfOpenState: 3
```

“backend\api-gateway\src\main\resources\application.yaml”

6.1.5 Client Application and Hardware Integration Phase

This section demonstrates the frontend architecture and native mobile hardware integrations utilizing the Capacitor framework.

Native Android Microphone Integration:

Proves deep hardware integration by showing how the Angular app requests native microphone permissions and streams voice data via Capacitor.

```
import { Injectable, inject, signal } from '@angular/core';
import { Capacitor } from '@capacitor/core';
import { SpeechRecognition } from '@capacitor-community/speech-recognition';
import { AlertController } from '@ionic/angular/standalone';
import { Subject } from 'rxjs';

type SpeechRecognitionInstance = {
  continuous: boolean;
  interimResults: boolean;
  lang: string;
  onstart: (() => void) | null;
  onspeechstart: (() => void) | null;
  onspeechend: (() => void) | null;
  onresult: ((event: { results: { transcript: string }[][] }) => void) | null;
```

```

onerror: (() => void) | null;
onend: (() => void) | null;
stop: () => void;
start: () => void;
};

type SpeechRecognitionConstructor = new () => SpeechRecognitionInstance;

@Injectable({
  providedIn: 'root'
})
export class VoiceRecognitionService {
  private alertCtrl: AlertController = inject(AlertController);
  private isNative: boolean = Capacitor.isNativePlatform();

  public isRecording: ReturnType<typeof signal<boolean>> = signal<boolean>(false);

  public transcriptionUpdate: Subject<{ text: string; isPartial: boolean; }> = new Subject<{
text: string; isPartial: boolean }>();

  private nativeListenerHandle: import('@capacitor/core').PluginListenerHandle | null = null;
  private nativeStateListenerHandle: import('@capacitor/core').PluginListenerHandle | null =
null;
  private speechTimeout: ReturnType<typeof setTimeout> | null = null;
  private recognition: SpeechRecognitionInstance | null = null;

  constructor() {
    this.initSpeechRecognition();
  }

  private clearSpeechTimeout(): void {
    if (this.speechTimeout) {

```

```

clearTimeout(this.speechTimeout);
this.speechTimeout = null;
}
}

private getSpeechRecognitionCtor(): SpeechRecognitionConstructor | null {
    const globalWindow: Record<string, unknown> = window as unknown as Record<string,
unknown>;
    const ctor: unknown = globalWindow['SpeechRecognition'] ??
globalWindow['webkitSpeechRecognition'];
    return ctor ? (ctor as SpeechRecognitionConstructor) : null;
}

private bindRecognitionHandlers(): void {
    if (!this.recognition) {
        return;
    }
    this.recognition.onstart = () => {
        this.isRecording.set(true);
        this.clearSpeechTimeout();
        this.speechTimeout = setTimeout(() => {
            if (this.isRecording() && this.recognition) {
                this.recognition.stop();
            }
        }, 4000);
    };
    this.recognition.onspeechstart = () => { this.clearSpeechTimeout(); };
    this.recognition.onspeechend = () => {
        this.clearSpeechTimeout();
        this.speechTimeout = setTimeout(() => {
            if (this.isRecording() && this.recognition) {
                this.recognition.stop();
            }
        }, 4000);
    };
}

```

```

    }
    }, 2000);
};

this.recognition.onresult = (event: { results: { transcript: string }[][] }) => {
    this.clearSpeechTimeout();
    const transcript: string = event.results[0][0].transcript;
    if (transcript) {
        this.transcriptionUpdate.next({ text: transcript, isPartial: false });
    }
};

this.recognition.onerror = () => {
    this.clearSpeechTimeout();
    this.isRecording.set(false);
};

this.recognition.onend = () => {
    this.clearSpeechTimeout();
    this.isRecording.set(false);
};
}

private initSpeechRecognition(): void {
    if (this.isNative) {
        return;
    }
    const ctor: SpeechRecognitionConstructor | null = this.getSpeechRecognitionCtor();
    if (!ctor) {
        return;
    }
    this.recognition = new ctor();
    this.recognition.continuous = false;
    this.recognition.interimResults = false;
    this.recognition.lang = 'en-US';
}

```

```

    this.bindRecognitionHandlers();
}

public async toggleRecording(): Promise<void> {
    if (this.isNative) {
        await this.toggleNativeRecording();
    } else {
        this.toggleBrowserRecording();
    }
}

private async checkNativePermissions(): Promise<boolean> {
    let permStatus: { speechRecognition: string } = await
SpeechRecognition.checkPermissions();
    if (permStatus.speechRecognition !== 'granted') {
        permStatus = await SpeechRecognition.requestPermissions();
    }
    if (permStatus.speechRecognition !== 'granted') {
        const alert: HTMLIonAlertElement = await this.alertCtrl.create({
            header: 'Permission Denied',
            message: 'Microphone and speech recognition permissions are required to capture voice
input.',
            buttons: ['OK'],
            cssClass: 'premium-alert'
        });
        await alert.present();
        return false;
    }
    return true;
}

private async checkNativeAvailability(): Promise<boolean> {

```

```

const avail: { available: boolean; } = await SpeechRecognition.available();
if (!avail.available) {
  const alert: HTMLIonAlertElement = await this.alertCtrl.create({
    header: 'Speech Recognition Unavailable',
    message: 'Speech recognition is not supported on this device.',
    buttons: ['OK'],
    cssClass: 'premium-alert'
  });
  await alert.present();
  return false;
}
return true;
}

private async setupNativeListeners(): Promise<void> {
  if (this.nativeListenerHandle) {
    await this.nativeListenerHandle.remove();
    this.nativeListenerHandle = null;
  }
  if (this.nativeStateListenerHandle) {
    await this.nativeStateListenerHandle.remove();
    this.nativeStateListenerHandle = null;
  }
  this.nativeListenerHandle = await SpeechRecognition.addListener('partialResults', (data: {
matches: string[] }) => {
    if (data.matches && data.matches.length > 0 && data.matches[0]) {
      this.transcriptionUpdate.next({ text: data.matches[0], isPartial: true });
    }
  });
  this.nativeStateListenerHandle = await SpeechRecognition.addListener('listeningState',
(data: { status: 'started' | 'stopped' }) => {
    if (data.status === 'stopped') {

```

```

    void this.stopNativeRecording();
  }
});
}

private async toggleNativeRecording(): Promise<void> {
  try {
    const isListening: { listening: boolean; } = await SpeechRecognition.isListening();
    if (isListening.listening || this.isRecording()) {
      await this.stopRecording();
      return;
    }
    const hasPermission: boolean = await this.checkNativePermissions();
    if (!hasPermission) {
      return;
    }
    const isAvailable: boolean = await this.checkNativeAvailability();
    if (!isAvailable) {
      return;
    }
    this.isRecording.set(true);
    await this.setupNativeListeners();
    await SpeechRecognition.start({ language: 'en-US', partialResults: true, popup: false });
    this.clearSpeechTimeout();
    this.speechTimeout = setTimeout(() => {
      if (this.isRecording()) {
        void this.stopNativeRecording();
      }
    }, 10000);
  } catch {
    this.isRecording.set(false);
    this.clearSpeechTimeout();
  }
}

```

```

    }
}

private async stopNativeRecording(): Promise<void> {
    this.clearSpeechTimeout();
    this.isRecording.set(false);
    try {
        await SpeechRecognition.stop();
    } catch {
        return;
    }
    await this.removeNativeListeners();
}

private async removeNativeListeners(): Promise<void> {
    if (this.nativeListenerHandle) {
        await this.nativeListenerHandle.remove();
        this.nativeListenerHandle = null;
    }
    if (this.nativeStateListenerHandle) {
        await this.nativeStateListenerHandle.remove();
        this.nativeStateListenerHandle = null;
    }
}

private toggleBrowserRecording(): void {
    if (!this.recognition) {
        this.handleSimulatedRecording();
        return;
    }
    if (this.isRecording()) {
        this.stopBrowserRecording();
    }
}

```

```

    } else {
        this.startBrowserRecording();
    }
}

private handleSimulatedRecording(): void {
    if (this.isRecording()) {
        this.isRecording.set(false);
        return;
    }
    this.isRecording.set(true);
    setTimeout(() => {
        if (this.isRecording()) {
            this.transcriptionUpdate.next({ text: '[Simulated premium voice input stream]', isPartial:
false });
            this.isRecording.set(false);
        }
    }, 3500);
}

private stopBrowserRecording(): void {
    this.clearSpeechTimeout();
    this.isRecording.set(false);
    try {
        this.recognition?.stop();
    } catch {
        return;
    }
}

private startBrowserRecording(): void {
    try {

```

```

        this.isRecording.set(true);
        this.recognition?.start();
    } catch {
        this.isRecording.set(false);
    }
}

public async stopRecording(): Promise<void> {
    if (this.isNative) {
        await this.stopNativeRecording();
    } else {
        this.stopBrowserRecording();
    }
}

public destroy(): void {
    this.clearSpeechTimeout();
    void this.stopRecording();
}
}

```

“frontend\src\app\domains\chat\data-access\voice-recognition.service.ts”

Text-to-Speech Engine:

Demonstrates the use of the browser's Web Speech Synthesis API to provide audible AI reflections, fulfilling accessibility requirements.

```

import { Injectable, inject, signal } from '@angular/core';

import { AudioVisualizerService } from '../audio-visualizer.service';

```

```

@Injectable({
  providedIn: 'root'
})
export class TextToSpeechService {
  public availableVoices: import('@angular/core').WritableSignal<SpeechSynthesisVoice[]>
= signal<SpeechSynthesisVoice[]>([]);
  public currentlySpeakingId: import('@angular/core').WritableSignal<string | null> =
signal<string | null>(null);

  private audioVisualizer: AudioVisualizerService = inject(AudioVisualizerService);

  constructor() {
    this.preCacheVoices();
  }

  private preCacheVoices(): void {
    if (typeof window !== 'undefined' && window.speechSynthesis) {
      let voicesLoaded: boolean = false;

      const loadVoices: () => void = () => {
        if (voicesLoaded) {return;}
        const voices: SpeechSynthesisVoice[] = window.speechSynthesis.getVoices().filter((v:
SpeechSynthesisVoice) => v.lang.startsWith('en'));
        if (voices.length > 0) {
          voicesLoaded = true;
          this.availableVoices.set(voices);
          if (window.speechSynthesis.removeEventListener) {
            window.speechSynthesis.removeEventListener('voiceschanged',
handleVoicesChanged);
          } else {
            window.speechSynthesis.onvoiceschanged = null;

```

```

    }
  }
};

const handleVoicesChanged: () => void = () => {
  loadVoices();
};

loadVoices();

if (!voicesLoaded) {
  if (window.speechSynthesis.addEventListener) {
    window.speechSynthesis.addEventListener('voiceschanged', handleVoicesChanged);
  } else {
    window.speechSynthesis.onvoiceschanged = handleVoicesChanged;
  }
}
}
}

public speakText(msgId: string, text: string): void {
  if (typeof window === 'undefined' || !window.speechSynthesis) {
    return;
  }

  if (this.currentlySpeakingId() === msgId) {
    window.speechSynthesis.cancel();
    this.currentlySpeakingId.set(null);
    this.audioVisualizer.restoreVolume();
    return;
  }
}

```

```

window.speechSynthesis.cancel();

const utterance: SpeechSynthesisUtterance = new SpeechSynthesisUtterance(text);

const voices: SpeechSynthesisVoice[] = window.speechSynthesis.getVoices();

const selectedVoice: SpeechSynthesisVoice = voices.find((v: SpeechSynthesisVoice) =>
v.name.toLowerCase().includes('google uk english female')) ||
  voices.find((v: SpeechSynthesisVoice) => v.name.toLowerCase().includes('google uk
english')) ||
  voices.find((v: SpeechSynthesisVoice) => v.name.toLowerCase().includes('samantha')) ||
  voices.find((v: SpeechSynthesisVoice) => v.name.toLowerCase().includes('zira')) ||
  voices.find((v: SpeechSynthesisVoice) => v.name.toLowerCase().includes('hazel')) ||
  voices.find((v: SpeechSynthesisVoice) => v.lang.startsWith('en') &&
v.name.toLowerCase().includes('female')) ||
  voices.find((v: SpeechSynthesisVoice) => v.lang.startsWith('en')) ||
  voices[0];

if (selectedVoice) {

  utterance.voice = selectedVoice;
}

utterance.rate = 0.95;
utterance.pitch = 1.05;

utterance.onend = () => {
  if (this.currentlySpeakingId() === msgId) {
    this.currentlySpeakingId.set(null);
  }
  this.audioVisualizer.restoreVolume();
}

```

```

};

utterance.onerror = () => {

    if (this.currentlySpeakingId() === msgId) {
        this.currentlySpeakingId.set(null);
    }
    this.audioVisualizer.restoreVolume();
};

this.currentlySpeakingId.set(msgId);

this.audioVisualizer.duckVolume(0.18);

setTimeout(() => {
    window.speechSynthesis.speak(utterance);
}, 100);
}

public cancel(): void {
    if (typeof window !== 'undefined' && window.speechSynthesis) {
        window.speechSynthesis.cancel();
        this.currentlySpeakingId.set(null);
        this.audioVisualizer.restoreVolume();
    }
}
}

```

“frontend\src\app\domains\chat\data-access\text-to-speech.service.ts”

Reactive State Management:

Illustrates advanced frontend architecture using RxJS Observables to handle asynchronous data streams and HTTP client requests.

```
import { HttpClient, HttpContext } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable, of, timeout } from 'rxjs';

import { REFLECTION_QUOTES } from '../../../core/constants/quotes.constants';
import { SKIP_CANCEL } from '../../../core/interceptors/cancel.interceptor';
import { ApiService } from '../../../core/services/api.service';

export interface ChatMessage {
  id: string;
  sender: 'user' | 'mirror';
  text: string;
  timestamp: Date;
  isTyping?: boolean;
  emotion?: string;
  primaryColor?: string;
  secondaryColor?: string;
  isCurrentSession?: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class ChatService {
  private http: HttpClient = inject(HttpClient);
  private apiSvc: ApiService = inject(ApiService);
```

```

    public getHistory(_email: string, cursor: string | null, size: number): Observable<{
messages: unknown[], hasMore: boolean, nextCursor: string | null }> {
    const cursorParam: string = cursor ? `&cursor=${cursor}` : "";
    return this.http.get<{ messages: unknown[], hasMore: boolean, nextCursor: string | null
}>(`${this.apiSvc.userMemory.HISTORY}?size=${size}${cursorParam}`);
    }

    public reflect(_email: string, prompt: string): Observable<{ reflection: string, emotion:
string }> {
    return this.http.post<{ reflection: string, emotion: string
}>(this.apiSvc.userMemory.REFLECT, prompt, {
        headers: {
            'Content-Type': 'text/plain'
        },
        context: new HttpContext().set(SKIP_CANCEL, true)
    }).pipe(
        timeout(30000)
    );
    }

    public getRandomQuote(): Observable<{ quote: string, author: string }> {
    const randomIdx: number = Math.floor(Math.random() *
REFLECTION_QUOTES.length);
    return of(REFLECTION_QUOTES[randomIdx]);
    }
}

```

“frontend\src\app\domains\chat\data-access\chat.service.ts”

6.2 Sample Screen Shots

This section provides a visual representation of the application's user interface, illustrating the overall design and user experience. The following screenshots demonstrate the core functionalities and interactive flows available to the user, from initial authentication to the main chat interface and account management. Each subsection below highlights a specific view and outlines the key user journeys associated with it.

6.2.1 Authentication & Onboarding Views

This collection of views handles all user identity workflows, from initial account creation and email verification to secure login and password recovery.

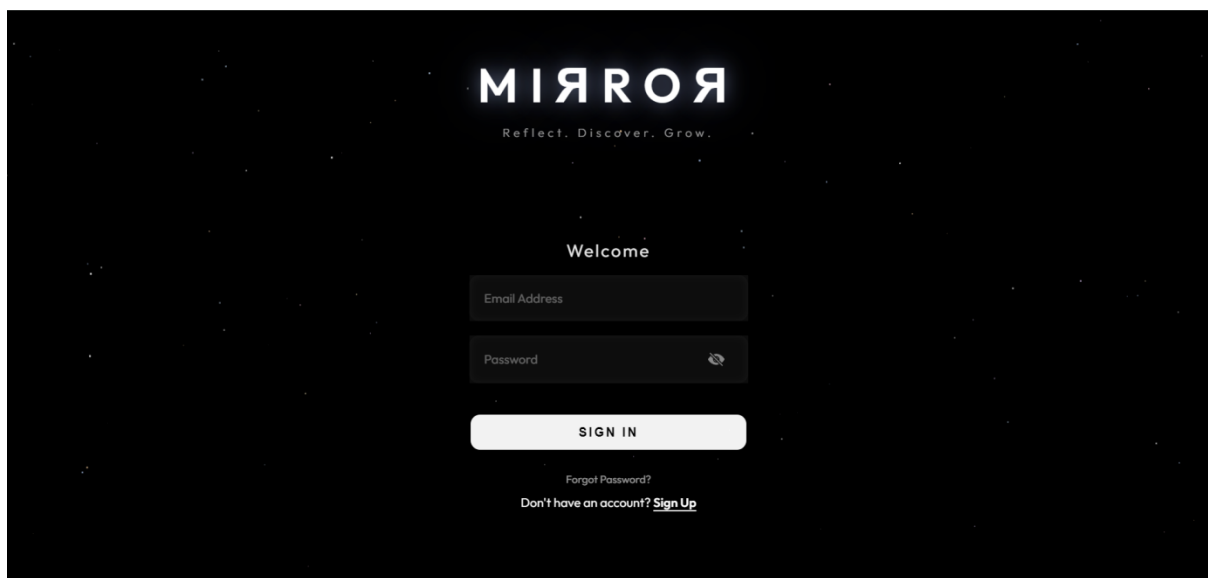
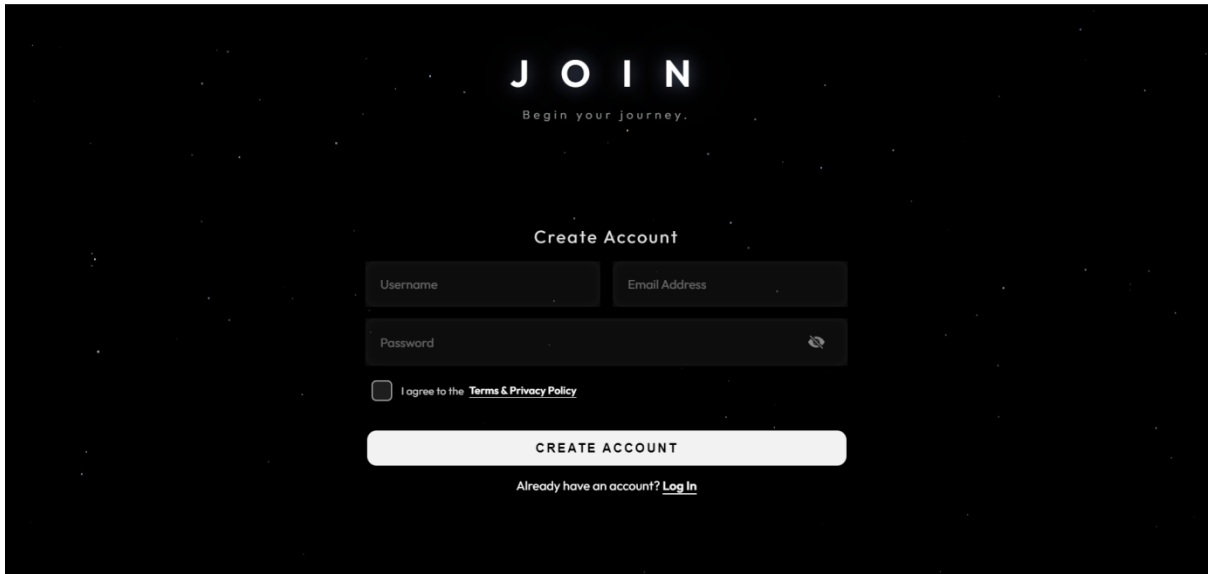


Figure 6.2.1: User Login Interface.



The image shows a dark-themed user interface for account registration. At the top, the word "JOIN" is displayed in large, white, spaced-out capital letters, with the tagline "Begin your journey." centered below it. The main heading "Create Account" is centered above the form fields. The form consists of three input fields: "Username", "Email Address", and "Password". The "Password" field includes a small eye icon for toggling visibility. Below the fields is a checkbox labeled "I agree to the Terms & Privacy Policy". A prominent white button labeled "CREATE ACCOUNT" is centered below the checkbox. At the bottom, a link "Already have an account? Log In" is displayed.

JOIN
Begin your journey.

Create Account

Username Email Address

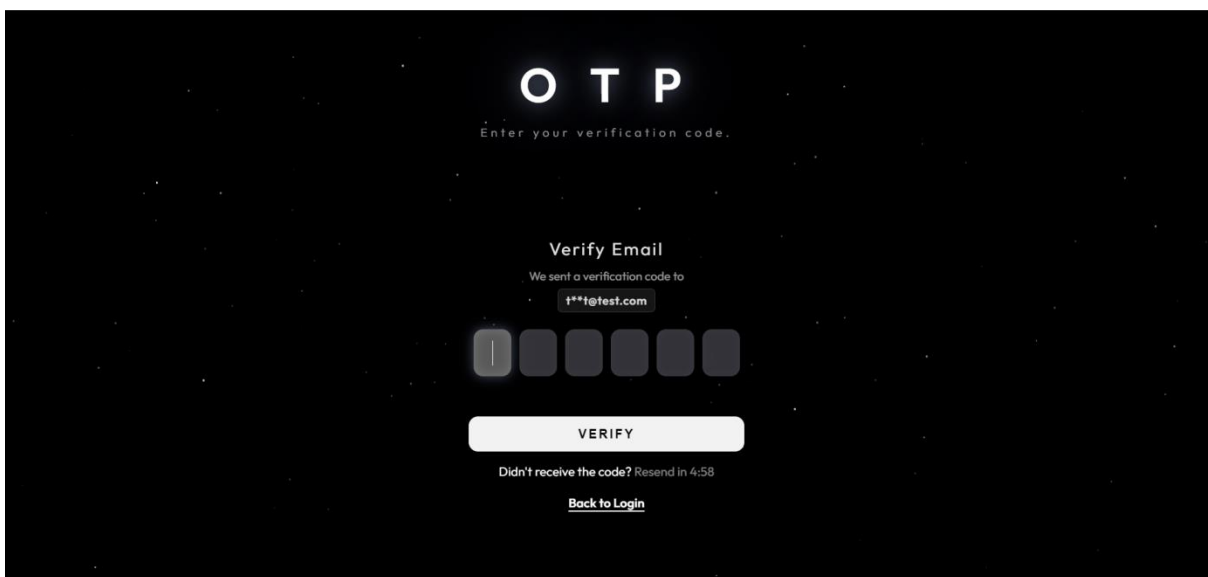
Password

☐ I agree to the Terms & Privacy Policy

CREATE ACCOUNT

Already have an account? [Log In](#)

Figure 6.2.2: Account Registration.



The image shows a dark-themed user interface for OTP verification. At the top, the letters "OTP" are displayed in large, white, spaced-out capital letters, with the instruction "Enter your verification code." centered below it. The heading "Verify Email" is centered above the verification details. Below this, it states "We sent a verification code to" followed by a masked email address "t**t@test.com". A row of six input boxes is provided for the verification code, with the first box containing a vertical cursor. A white button labeled "VERIFY" is centered below the input boxes. At the bottom, a link "Didn't receive the code? Resend in 4:58" is displayed, followed by a link "Back to Login".

OTP
Enter your verification code.

Verify Email

We sent a verification code to
t**t@test.com

|

VERIFY

Didn't receive the code? Resend in 4:58

[Back to Login](#)

Figure 6.2.3: OTP Verification.

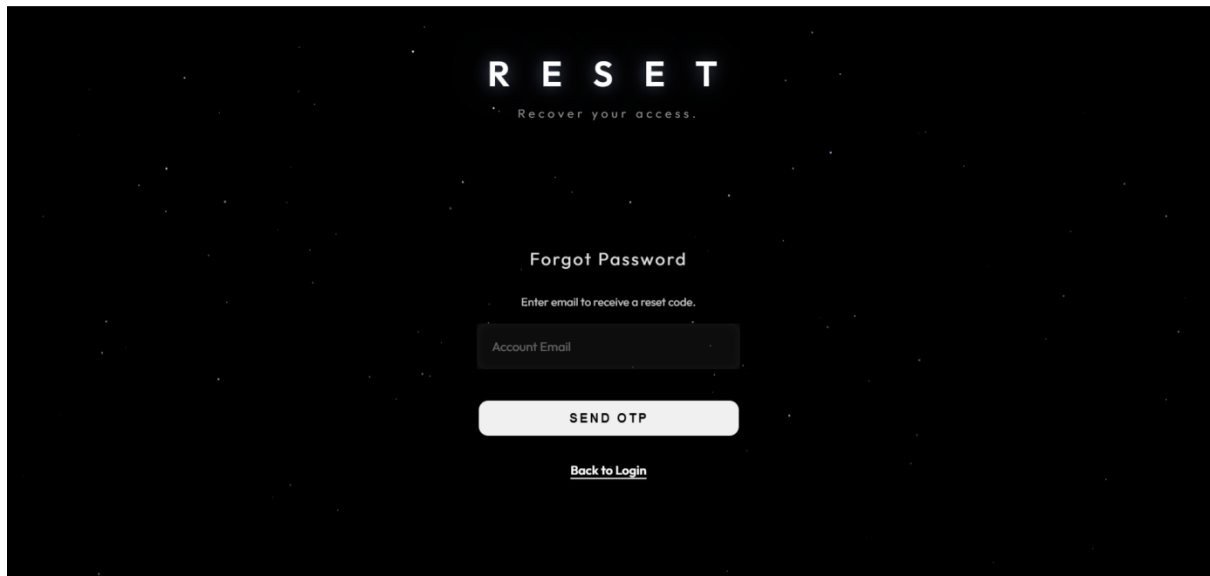


Figure 6.2.4: Password Reset Request Interface.

6.2.2 Main Dashboard & Interaction Views

Upon successful login, users are greeted with an ambient, minimalist loading or introductory screen that centers on the user's presence, establishing the app's calming aesthetic.



Figure 6.2.5: Ambient Welcome Screen.

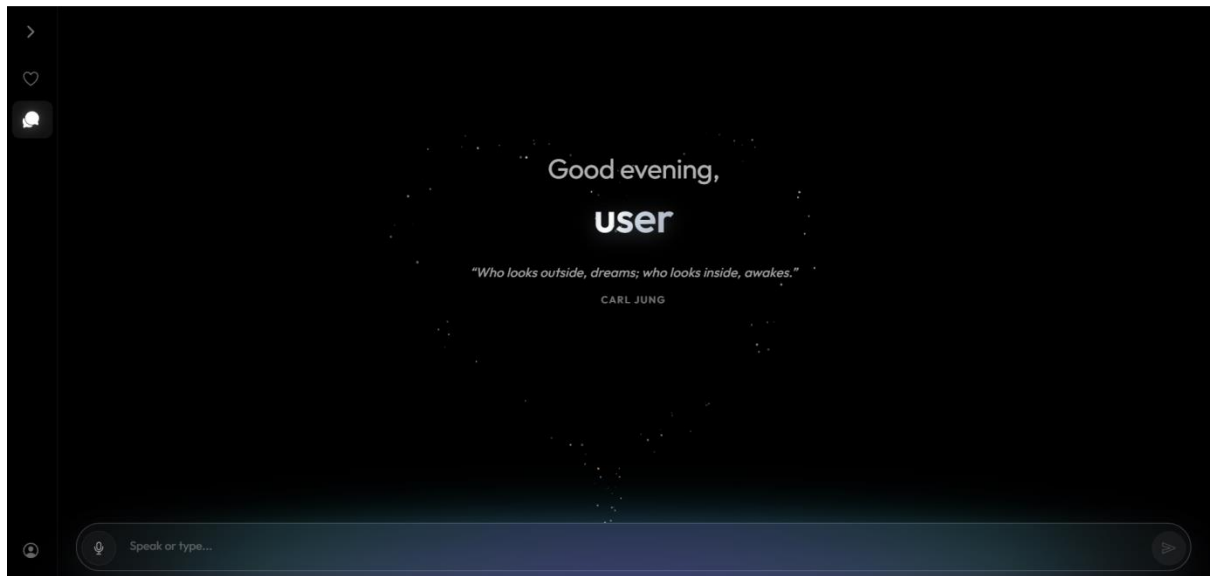


Figure 6.2.6: Chat interface with Daily Quote for first-time users or when a new day starts.

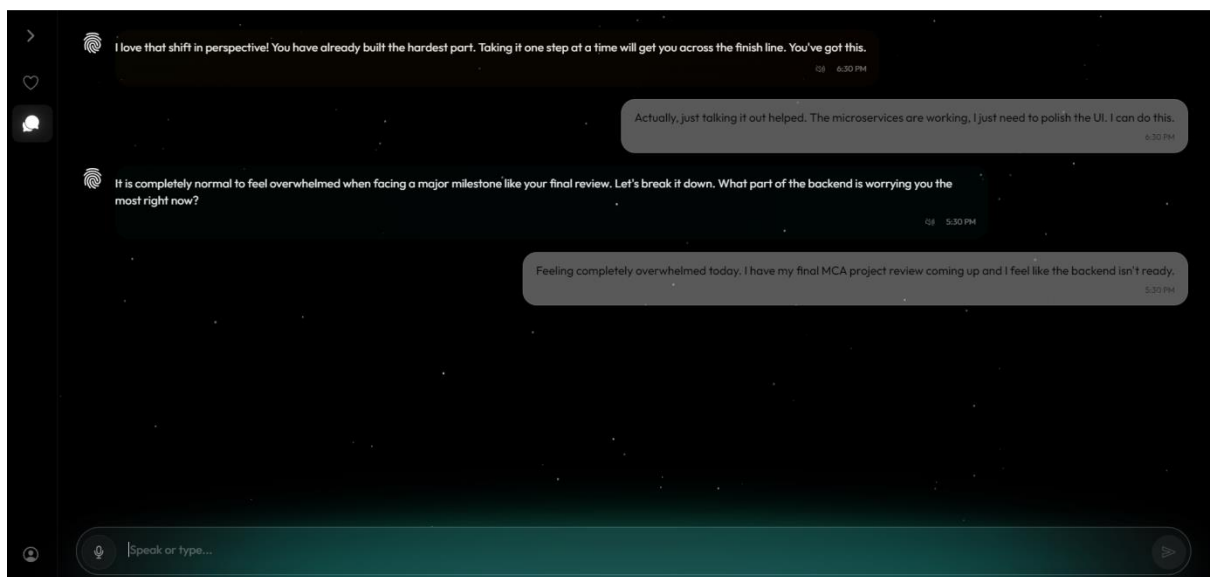


Figure 6.2.7: Active Chat Interface.

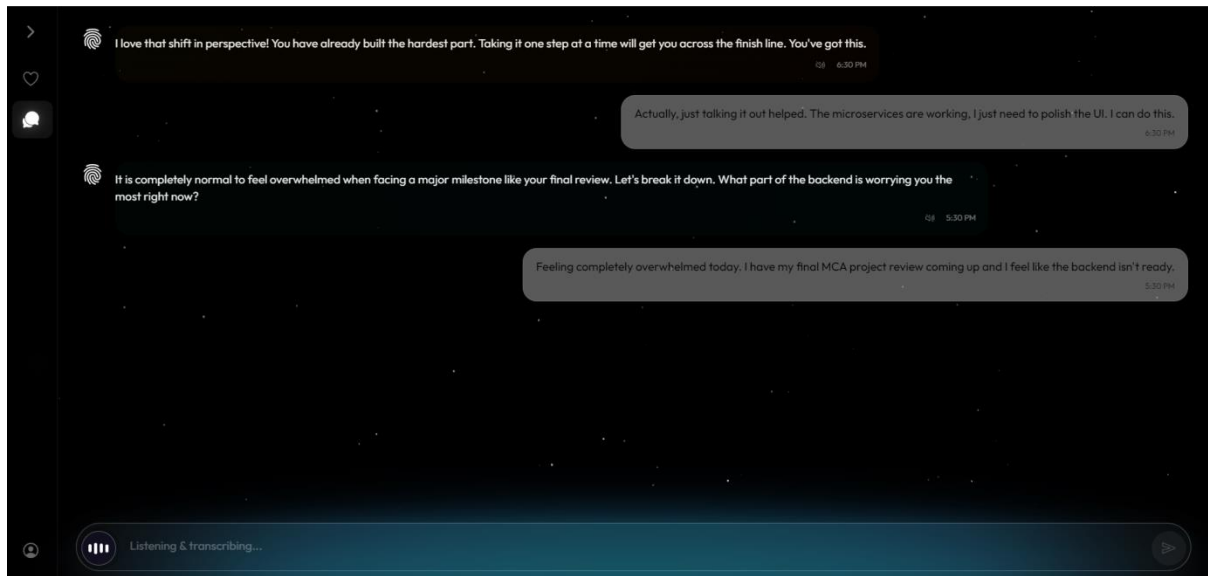


Figure 6.2.8: Voice Transcription and Chat History.

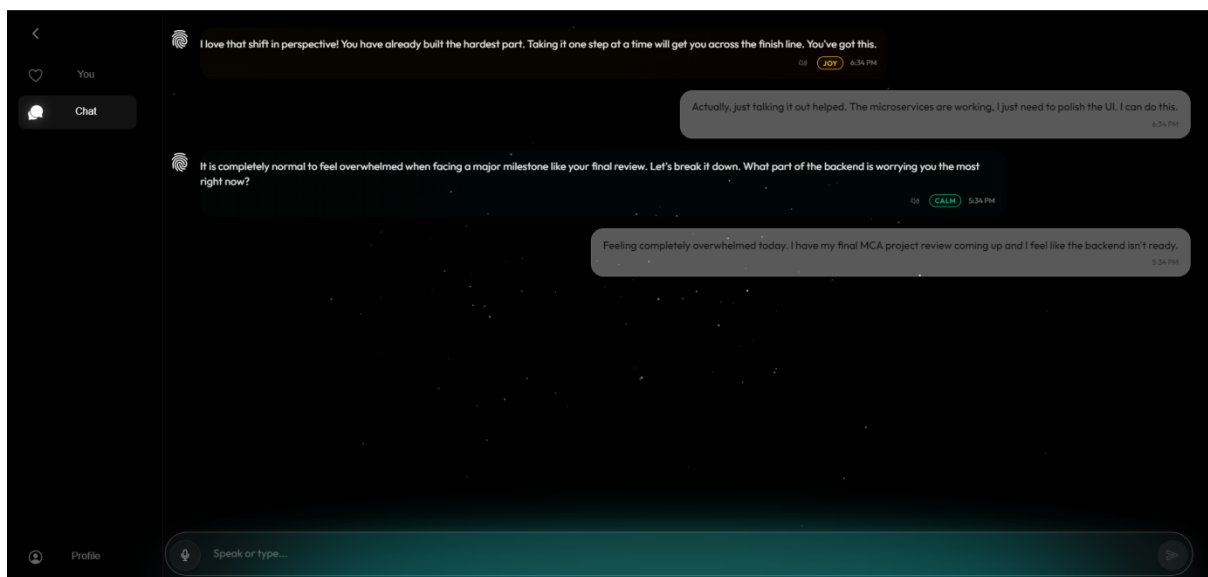


Figure 6.2.9: Expanded Sidebar Navigation Menu.

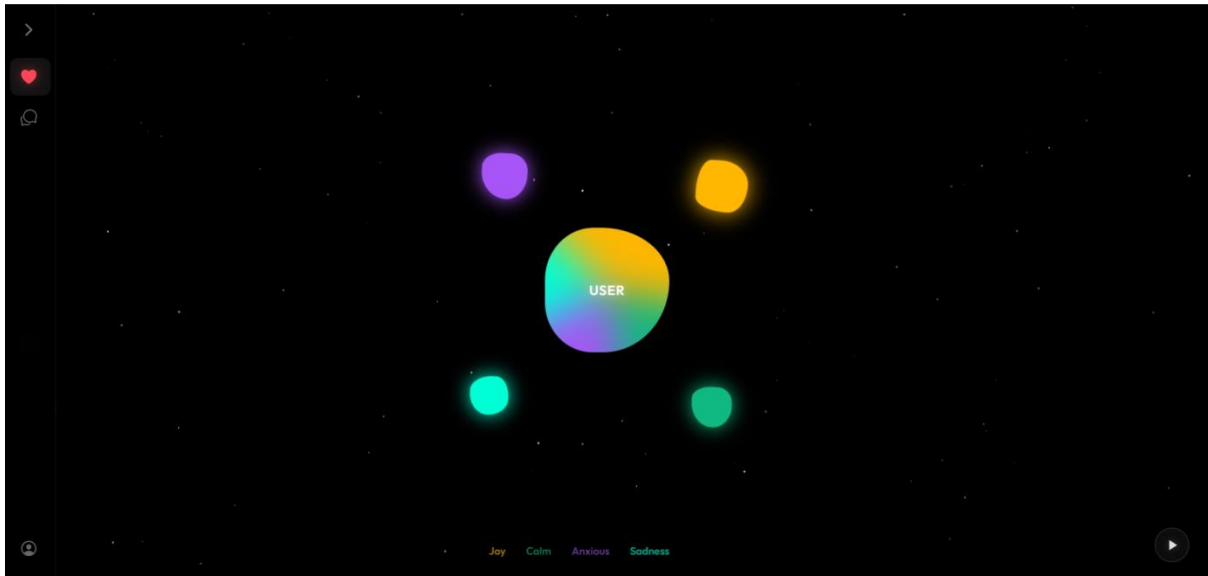


Figure 6.2.10: Mood and Emotional State Tracker.

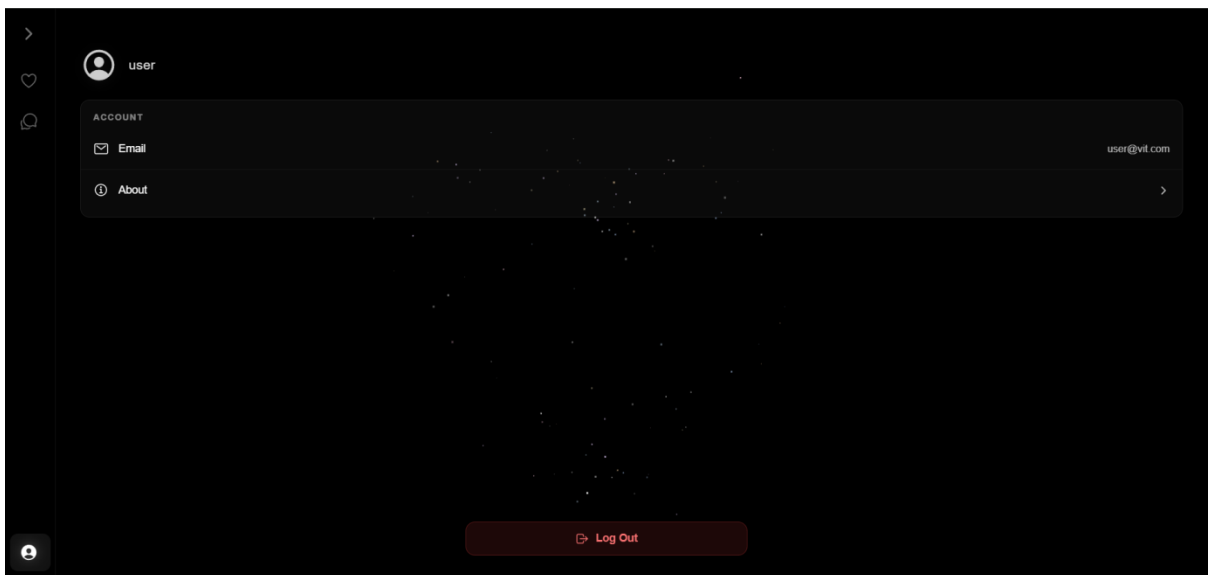


Figure 6.2.11: User Profile and Account Settings.

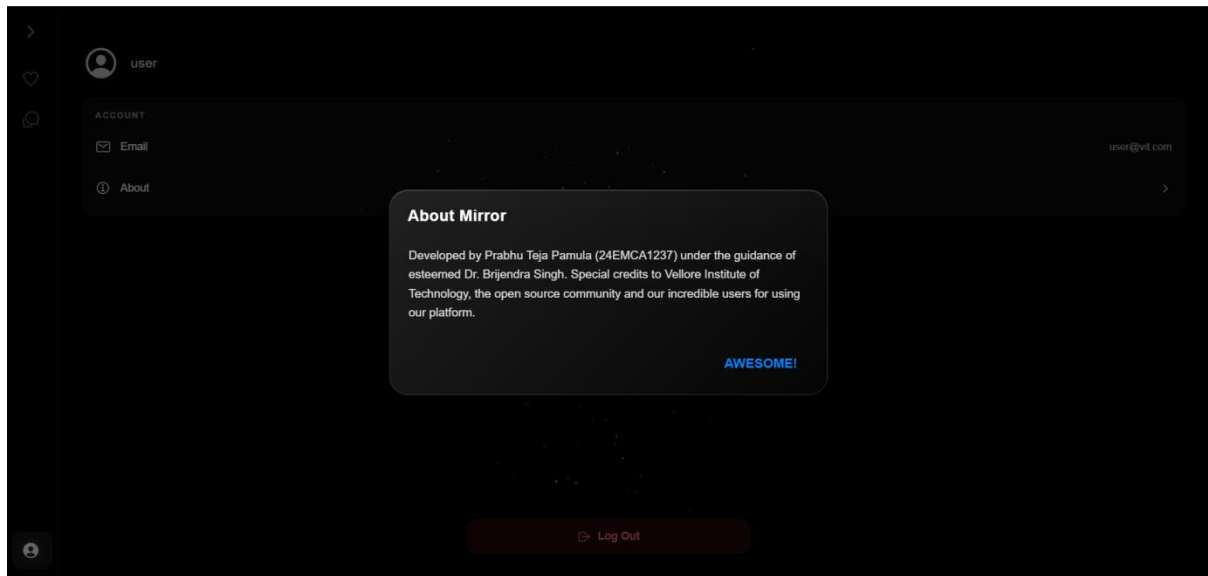


Figure 6.2.12: Project Acknowledgements and Appreciation Modal.

6.2.3 Administrator Views

Users with administrative privileges have access to an expanded profile menu that includes an exclusive "Admin View" portal for system management.

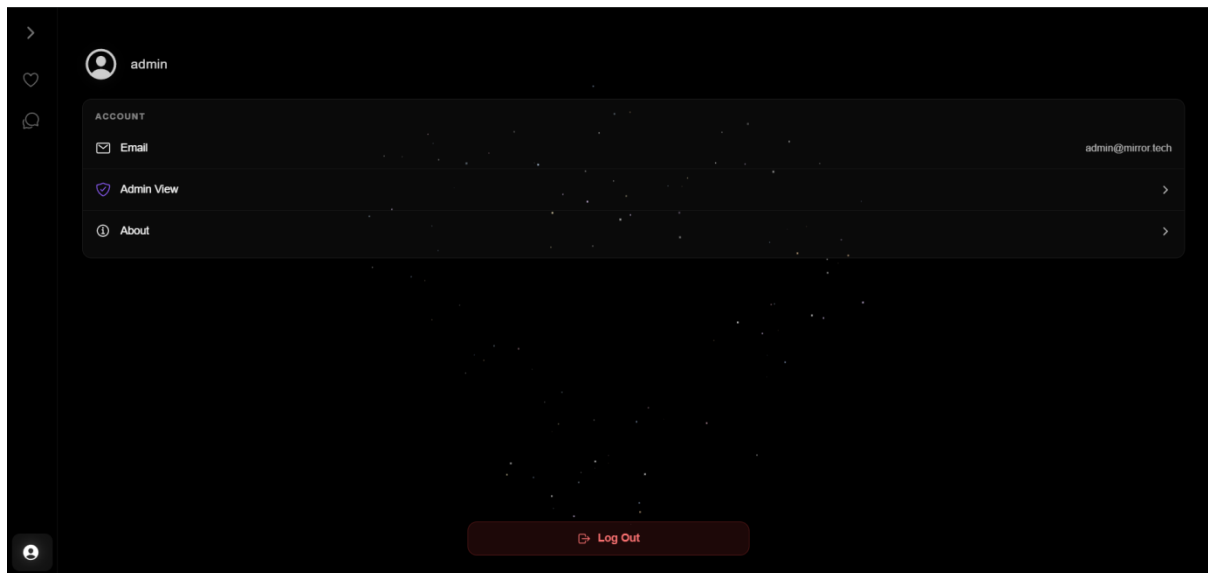


Figure 6.2.13: Administrator Profile Interface.

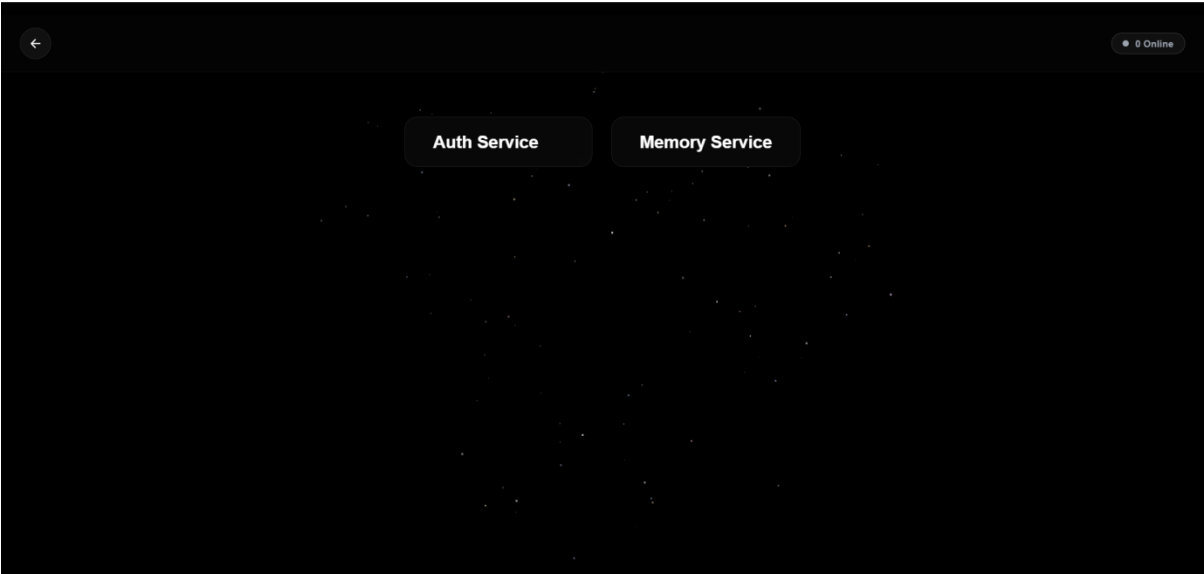


Figure 6.2.14: Admin Service Selection Interface.

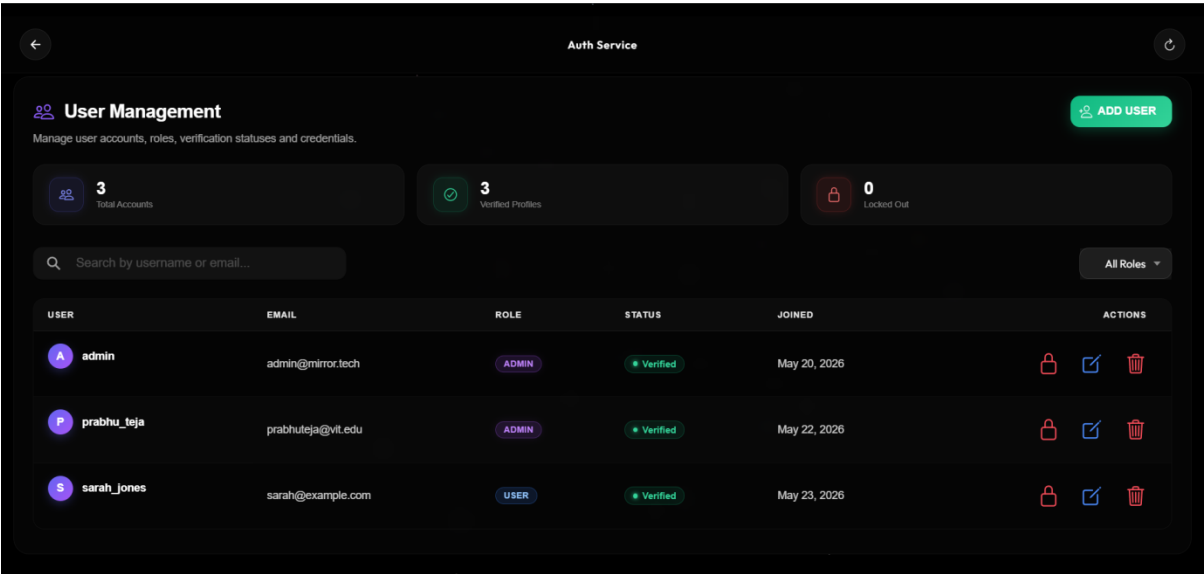


Figure 6.2.15: User Management Dashboard.

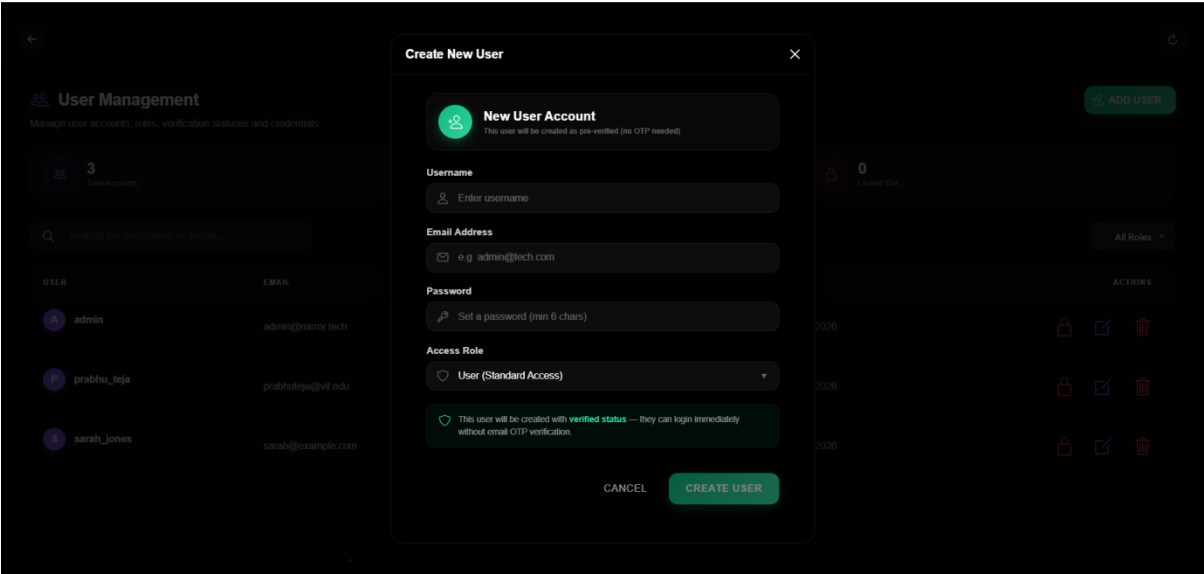


Figure 6.2.16: Create New User Modal Interface.

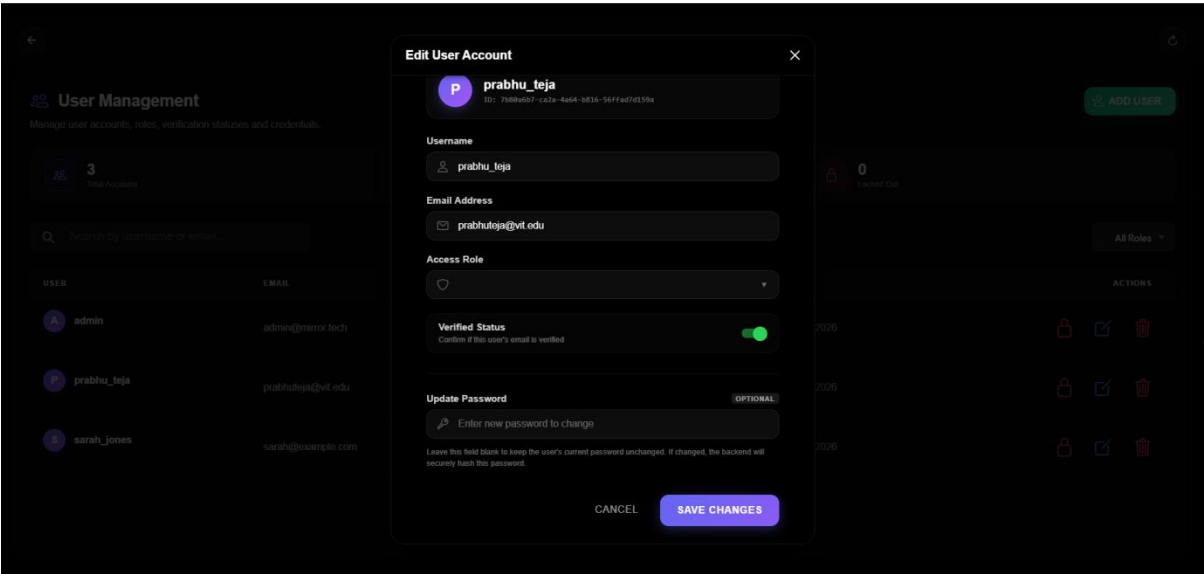


Figure 6.2.17: Edit User Account Modal Interface.

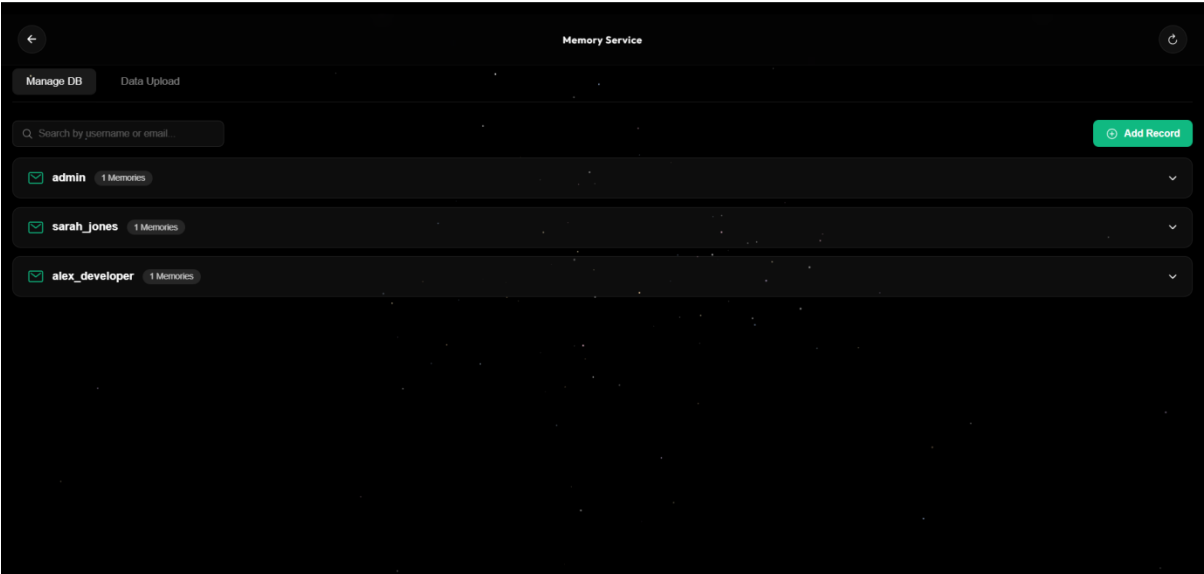


Figure 6.2.18: Memory Service Database Management Dashboard.

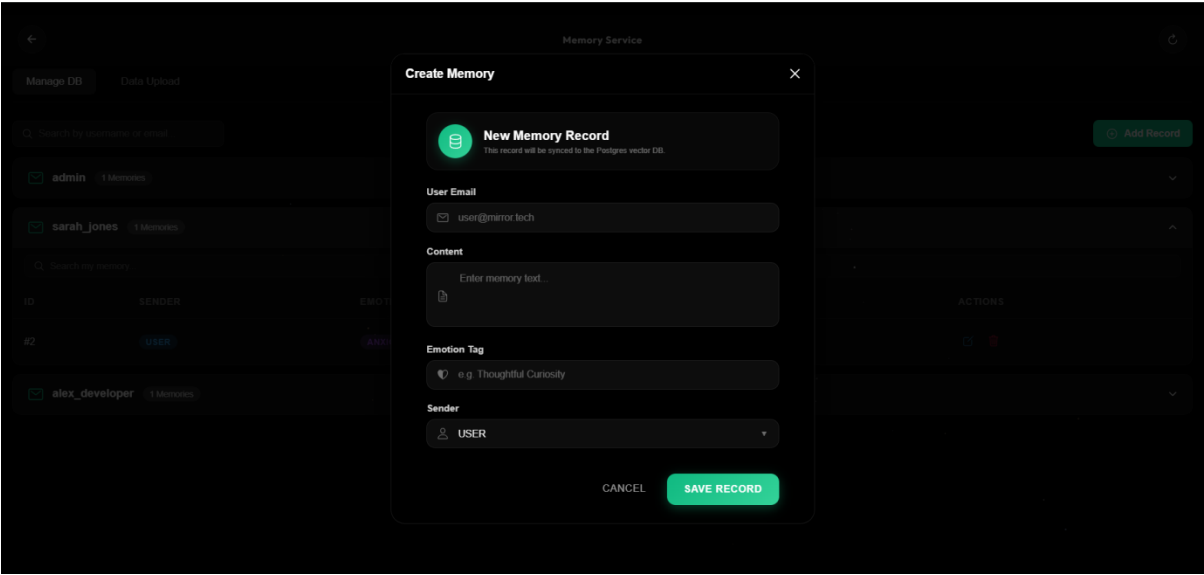


Figure 6.2.19: Create New Memory Record Modal.

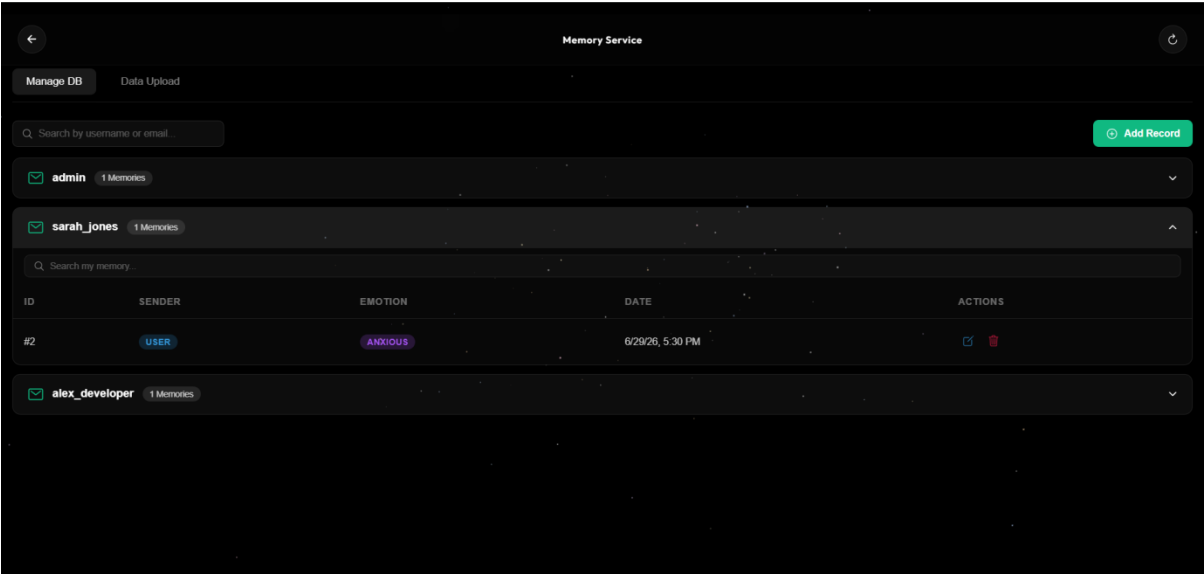


Figure 6.2.20: Expanded User Memory Records View.

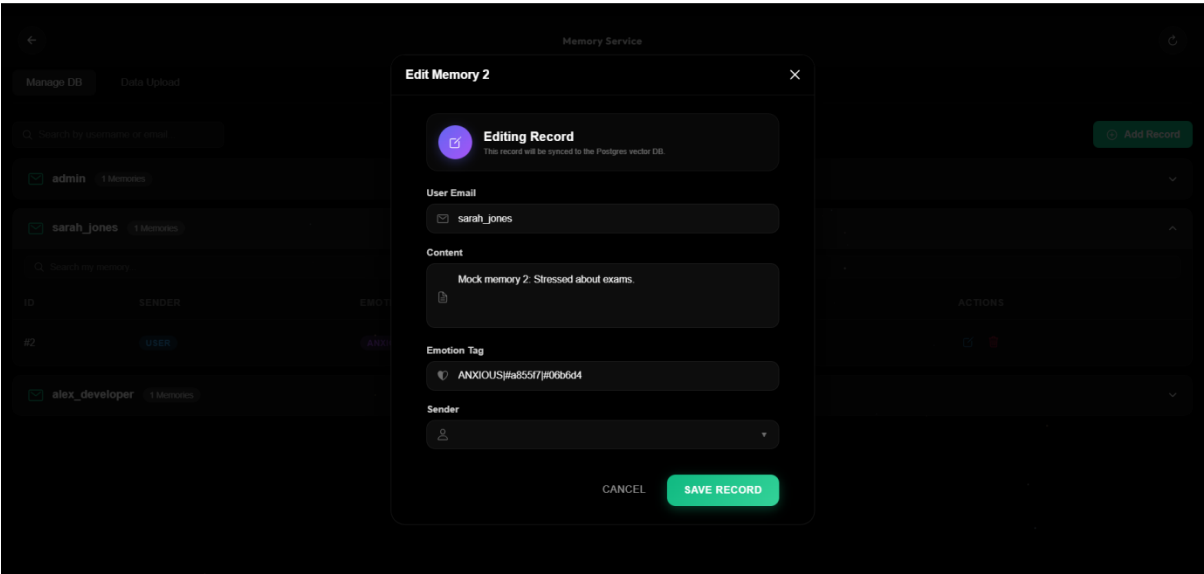


Figure 6.2.21: Edit Memory Record Modal.

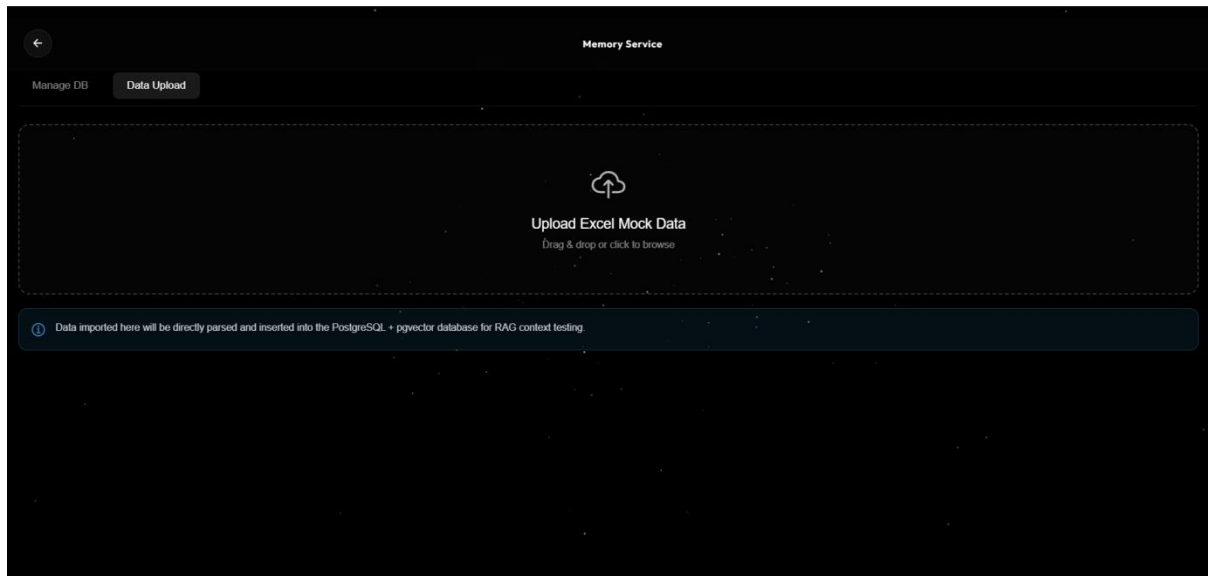


Figure 6.2.22: Memory Data Bulk Upload Interface.

6.2.4 Application Exit

When a user or administrator chooses to sign out from the Profile menu, a confirmation modal appears to prevent accidental logouts, securely terminating the current session upon confirmation.

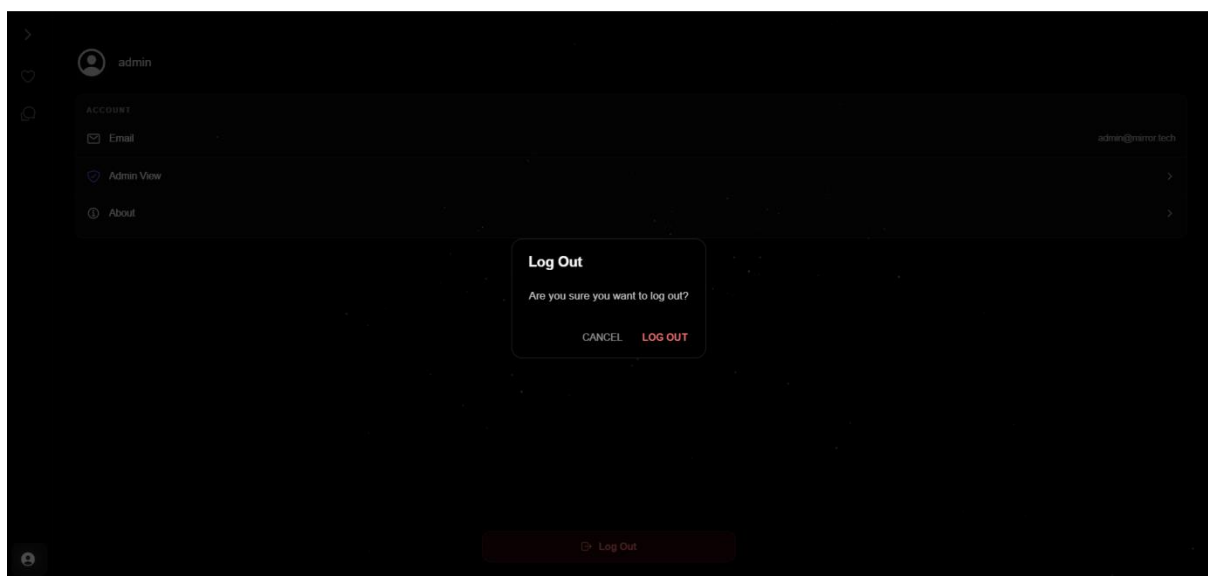


Figure 6.2.23: Log Out Confirmation Modal.

CHAPTER 7

COST ANALYSIS, RESULTS, AND DISCUSSION

This chapter provides a comprehensive evaluation of the MIRROR platform, focusing on the financial sustainability of the implemented architecture and a critical assessment of the system's performance. By analyzing the resource allocation strategy and validating the platform's functional outcomes against the requirements defined in Chapter 3, this chapter offers a final, empirical verification of the project's viability. The subsequent sections present a detailed cost-benefit analysis and a critical discussion of the system's performance, stability, and adherence to modern software engineering standards.

7.1 Cost Analysis

The infrastructure strategy was designed to minimize recurring operational expenditures (OPEX) while maintaining high-availability standards suitable for a production-ready application. This was achieved by leveraging educational partnerships, free-tier offerings from industry-leading service providers, and efficient client-side processing.

Service Category	Provider	Cost Basis	Implementation Details
Domain Registration	.TECH (via GitHub)	\$0.00 (1 Year)	Used the .tech domain extension for professional branding (projectmirror.tech).
Frontend Hosting	Vercel (Free Tier)	\$0.00 / month	Global edge network delivery and automated CI/CD deployment directly from the GitHub repository.
Backend Infrastructure	Azure Educational	\$0.00 / month	Hosted Java Spring Boot microservices (API

	Credit		Gateway, Auth, Memory) and messaging brokers (RabbitMQ/Redis) utilizing student cloud credits.
Database Services	Neon.tech (Free Tier)	\$0.00 / month	Serverless Postgres infrastructure with the pgvector extension enabled for semantic memory storage.
AI & LLM Services	Groq & Google Gemini APIs	\$0.00 (Developer Tiers)	Groq (LPU Inference Engine) is utilized for ultra-low latency conversational responses, while Gemini provides robust capabilities for deep emotional analytics and complex processing. Both operate within their respective free tiers.
Analytics	Firebase	\$0.00 / month	Integrated via Capacitor for cross-platform user telemetry on the free Spark plan.
Email Delivery	Brevo (SMTP Relay)	\$0.00 / month	Utilized by the Auth

			Service to handle transactional emails (e.g., registrations, password resets) within the free tier limit of 300 emails/day.
Speech Processing	Native Web Speech API	\$0.00 / month	Zero-cost Text-to-Speech (TTS) achieved by offloading audio synthesis entirely to the client device's native OS, avoiding costly cloud TTS APIs.

Economic Viability Summary

The total monthly recurring cost for the platform is currently \$0.00, enabled by the strategic utilization of educational grants, free-tier service allowances, and decentralized processing (client-side TTS). This configuration demonstrates that a sophisticated, AI-driven microservices architecture can be deployed and maintained with zero financial capital, provided the architectural design prioritizes efficient resource consumption and modular service hosting.

7.2 Results and Discussion

The implementation of the MIRROR platform successfully met the objectives established in the initial requirements analysis. By integrating Groq's high-speed inference engine alongside Gemini, the system achieved sub-second latency for conversational operations, and the RAG-based cognitive core demonstrated high accuracy in retrieving contextually relevant user memories and inferring emotional states.

Key findings from the implementation include:

- **Architectural Scalability:** The adoption of a Spring Boot microservices approach facilitated independent deployment cycles. This allowed for rapid iterations on the AI-processing engine (Memory Service) without disrupting the core identity and routing mechanisms (Auth Service and API Gateway).
- **Resilience and Stability:** The integration of Resilience4j circuit breakers and Redis-backed rate limiting within the API Gateway proved highly effective. These mechanisms successfully protected backend system components from cascading failures and prevented API abuse under simulated stress during validation phases.
- **Vector Search Efficiency:** The utilization of pgvector for HNSW (Hierarchical Navigable Small World) indexing in the Neon Postgres database provided a significant performance advantage for semantic search. This ensured that the database latency required for Retrieval-Augmented Generation (RAG) integration remained within acceptable limits for a real-time chat interface.
- **Client-Side Processing Efficacy:** Offloading Text-to-Speech (TTS) capabilities to the native Web Speech API not only eliminated cloud processing costs but significantly improved perceived application speed by removing the network latency associated with downloading generated audio files.

In conclusion, the results suggest that while the platform is currently optimized for high performance and zero-cost operation, future scalability to thousands of concurrent users would necessitate a transition to production-tier cloud environments and paid API tiers. However, the technical proof-of-concept achieved here validates the feasibility of a privacy-first, personal-scale reflection tool leveraging state-of-the-art LLM architectures.

CHAPTER 8

SUMMARY

The development of the MIRROR platform represents the culmination of a rigorous engineering effort to build a secure, private, and intelligent reflection system. By integrating advanced RAG pipelines with a modular microservices architecture, this project successfully addressed the need for a personalized cognitive assistant that operates with high precision and low latency. This summary synthesizes the project's journey, confirms the fulfillment of initial research objectives, acknowledges the inherent technical boundaries of the current implementation, and provides a roadmap for future expansion.

8.1 Project Overview

The MIRROR platform was conceptualized as an AI-augmented memory and reflection tool designed to securely catalog and analyze personal interaction data. Leveraging a robust full-stack architecture comprising a Java Spring Boot backend, an Angular/Ionic cross-platform frontend, PostgreSQL with pgvector, and state-of-the-art LLMs, the platform demonstrates that high-performance, intelligent systems can be architected with an emphasis on data privacy and zero-cost operational overhead.

8.2 Achievement of Objectives

The project successfully realized the core requirements established in the foundational design phases:

- **Secure Authentication:** Implemented stateless JWT management and Role-Based Access Control (RBAC) via Spring Security to ensure data integrity, user authorization, and secure routing through the API Gateway.
- **Intelligent Retrieval:** Successfully deployed an HNSW-indexed vector search pipeline for high-speed, semantically relevant memory retrieval that mimics cognitive recall.
- **Cognitive Integration:** Achieved real-time AI reflection using a decoupled event-driven architecture (via RabbitMQ) and high-speed LPU inference (via Groq),

supported by Gemini for deep analytical processing.

- **Cross-Platform Accessibility:** Utilized Angular alongside Ionic and Capacitor to deploy a unified codebase, successfully delivering the application as both a Progressive Web App (PWA) and a native Android application.
- **System Administration & Observability:** Developed a comprehensive, RBAC-secured Admin Dashboard allowing for centralized management of users, synthetic data ingestion, and memory vector oversight.
- **DevOps & Automation:** Established automated CI/CD deployment pipelines (e.g., Vercel edge networks) to ensure rapid, reliable, and continuous delivery of frontend updates.
- **Client-Side Optimization:** Capitalized on native OS Web Speech and Speech Recognition APIs to offload processing to the client device, effectively eliminating latency and cloud costs associated with audio synthesis.
- **Zero-Capital Scalability:** Validated the economic viability of the platform through strategic utilization of educational grants, free-tier cloud infrastructure, and optimized client-side processing.

8.3 Limitations and Challenges

Despite the successful implementation, several technical boundaries remain:

- **LLM Dependency:** The system relies on external API providers (Groq/Gemini), creating a dependency on third-party availability, rate-limiting policies, and potential privacy considerations regarding cloud processing.
- **Infrastructure Complexity & Cold Starts:** While microservices offer scalability, they introduced challenges in distributed tracing (requiring Zipkin) and inter-service state synchronization. Furthermore, relying entirely on free-tier and educational cloud environments often introduces "cold start" latencies after periods of inactivity.
- **Semantic Limitations:** The current embedding model, while efficient, faces limitations

in interpreting highly nuanced, long-term complex emotional contexts or resolving highly ambiguous semantic queries without further fine-tuning.

8.4 Future Research Directions

The MIRROR platform provides a robust foundation for continued development:

- **Local LLM Integration:** Transitioning to self-hosted, open-source models (e.g., Llama 3) to achieve absolute data sovereignty, eliminating external API dependencies and ensuring all reflections are processed on privately owned hardware.
- **Advanced Analytics:** Extending the emotional dashboard to include predictive longitudinal mood tracking and sentiment trend analysis over extended periods.
- **Enhanced Hardware Integration:** Exploring deeper integration with biometric data or wearable device sensors (via Capacitor plugins) to provide real-time physiological context (e.g., heart rate, stress levels) for AI reflections.
- **Continuous Voice Modality:** Evolving the current speech-to-text and text-to-speech features into a full-duplex, continuous conversational voice agent, moving beyond turn-based text interactions.

CHAPTER 9

REFERENCES

- [1] Spring Framework, "Spring Boot Reference Documentation," VMware, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/index.html> (accessed Jun. 30, 2026).
- [2] Angular, "Angular Documentation," Google, 2026. [Online]. Available: <https://angular.dev> (accessed Jun. 30, 2026).
- [3] Ionic Framework, "Capacitor Documentation," 2026. [Online]. Available: <https://capacitorjs.com/docs> (accessed Jun. 30, 2026).
- [4] PostgreSQL Global Development Group, "pgvector: Open-source vector similarity search for Postgres," 2026. [Online]. Available: <https://github.com/pgvector/pgvector> (accessed Jun. 30, 2026).
- [5] Google AI Studio, "Gemini API Documentation," Google, 2026. [Online]. Available: <https://ai.google.dev/docs> (accessed Jun. 30, 2026).
- [6] Groq, "GroqCloud API Documentation," Groq Inc., 2026. [Online]. Available: <https://console.groq.com/docs/overview> (accessed Jun. 30, 2026).
- [7] RabbitMQ, "RabbitMQ Documentation," VMware, 2026. [Online]. Available: <https://www.rabbitmq.com/documentation.html> (accessed Jun. 30, 2026).
- [8] Neon, "Neon Serverless Postgres Documentation," 2026. [Online]. Available: <https://neon.tech/docs/introduction> (accessed Jun. 30, 2026).
- [9] Redis, "Redis Documentation," Redis Ltd., 2026. [Online]. Available: <https://redis.io/docs/> (accessed Jun. 30, 2026).
- [10] Resilience4j, "Resilience4j Documentation - Circuit Breaking and Rate Limiting," 2026. [Online]. Available: <https://resilience4j.readme.io/docs> (accessed Jun. 30, 2026).
- [11] Zipkin, "Zipkin Distributed Tracing System," 2026. [Online]. Available: <https://zipkin.io/pages/documentation.html> (accessed Jun. 30, 2026).

[12] Firebase, "Firebase Documentation," Google, 2026. [Online]. Available: <https://firebase.google.com/docs> (accessed Jun. 30, 2026).

[13] NgRx, "NgRx Documentation - Reactive State for Angular," 2026. [Online]. Available: <https://ngrx.io/docs> (accessed Jun. 30, 2026).

[14] Mozilla Developer Network (MDN), "Web Speech API Documentation," 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API (accessed Jun. 30, 2026).

[15] JSON Web Token, "JWT (JSON Web Tokens) documentation," Auth0, 2026. [Online]. Available: <https://jwt.io/introduction> (accessed Jun. 30, 2026).

[16] Brevo, "Brevo API Documentation for Transactional Emails," 2026. [Online]. Available: <https://developers.brevo.com> (accessed Jun. 30, 2026).

APPENDIX A

This appendix provides supplementary technical documentation for the MIRROR platform, including deployment instructions, core database schemas, and microservice routing configurations. This information serves as a technical reference to prove the system's architecture and replicability.

A.1 System Deployment Guide

The system utilizes a combination of Docker containers for infrastructure and Maven wrapper (mvnw) for the microservices. A custom PowerShell script is provided to fully automate the local startup sequence.

Prerequisites

- Docker Engine: Version 24.0.0+ (Required for PostgreSQL, Redis, RabbitMQ, Zipkin).
- Node.js & npm: Node 20.x+ (Required for the Angular/Ionic frontend).
- Java JDK: JDK 21 (Required for compiling Spring Boot microservices).
- Ionic CLI (Optional): Required if building the native Android application via Capacitor.

Automated Build and Run

1. Clone the Repository: Execute `git clone https://github.com/prabhu-tejaa/MIRROR` and navigate into the root directory.
2. Environment Variables: Before running the script, open `start-local-dev.ps1` and replace the placeholder dummy strings (e.g., `YOUR_GEMINI_API_KEY_HERE`) with your actual development keys and database connection strings.
3. Execute Startup Script: Run the following command from the project root:

```
.\start-local-dev.ps1
```

Note: This script will automatically start the Docker Compose infrastructure, set local environment defaults, and spawn separate terminal windows running the API Gateway (:8060), Auth Service (:8080), Memory Service (:8081), and Frontend (:4200).

A.2 Core AI Database Schema

To support the Retrieval-Augmented Generation (RAG) cognitive core, the system relies on the pgvector extension within PostgreSQL. Below is the primary migration schema (V1__init.sql) automatically executed via Flyway to initialize the high-dimensional vector memory storage:

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS memories (
  id BIGSERIAL PRIMARY KEY,
  user_id VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  emotion VARCHAR(50) NOT NULL,
  sender VARCHAR(50) DEFAULT 'user',
  embedding vector(768), -- Stores Gemini/Groq embeddings
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS memories_embedding_idx
ON memories USING hnsw (embedding vector_cosine_ops);
```

A.3 Microservice API Routing Table

The system employs a Spring Cloud API Gateway running on port 8060. It acts as the central entry point, handling CORS, Resilience4j Circuit Breaking, and Redis-backed Rate Limiting before securely routing requests to the internal microservices.

External Endpoint Path	Target Microservice	Internal Port	Primary Responsibility
/api/auth/**	Auth Service	8080	Handles user registration, JWT generation, OTP validation, and password resets via Brevo SMTP.
/api/memory/**	Memory Service	8081	Handles core AI chat

			interactions, emotional analytics parsing, and RAG vector searches.
/api/admin/memory/**	Memory Service	8081	RBAC-secured endpoints for administrators to inject synthetic data and oversee user memory banks.

A.4 System Access & Observability

Once the startup script completes and containers are operational, the following observability and management ports are exposed for debugging:

- Zipkin Tracing Dashboard: <http://localhost:9411> (For monitoring distributed request flows and microservice latency bottlenecks).
- RabbitMQ Management UI: <http://localhost:15672> (For monitoring asynchronous event queues and messaging health).